

Tracking Data and the Big Data Bowl

Every player, ten times a second: how to read the NFL’s tracking files, turn dots into football (separation, the pocket, motion, safety rotation), and what the Big Data Bowl has taught the league

A broadcast comes back from a commercial with a replay. The receiver runs a comeback, the ball arrives, and the screen freezes on a glowing ring around him: *separation 3.1 yards*. Later the same broadcast shows a safety’s path as a bright line sweeping from one hash to the middle of the field the instant the ball was snapped. Where do those numbers and lines come from, how much should you trust them, and could you make them yourself? Yes, with about a hundred lines of Python. This chapter shows how. It also builds, for real, the safety-rotation chart that Disguise, Rotation, and Split-Field Coverage showed you as a finished picture.

You’ll be able to:

- Read a tracking file: say what one row is, what `x`, `y`, `s`, `a`, `dis`, `o` and `dir` mean, and where the point (0, 0) is on the field.
- Flip every play so the offense moves left to right, and line thousands of plays up on the snap or on the throw.
- Draw and animate any play from tracking frames with `gridiron`, the same library that draws every diagram in this book.
- Compute separation, pocket area, motion at the snap and safety rotation from raw coordinates, and say what each number misses.
- Sketch three projects of your own: a man/zone classifier, a route classifier and a disguise measure.
- Recognise the Big Data Bowl’s themes since 2019, and what the winning entries had in common.

You’ll need

- **Tendencies: PROE, Personnel, Motion, and Coverage: coverage charting** (people watching film and labelling each play’s coverage, the source of the coverage columns in `nflverse` data and in the Big Data Bowl files) and the **neutral situation** filter. This chapter replaces “what someone charted” with “where everyone actually was.”
- **Disguise, Rotation, and Split-Field Coverage: disguise** (showing one coverage and playing another), **post-snap rotation** (safeties changing the shell after the snap) and **late rotation** (moving only at the snap, so the quarterback cannot see it coming). The rotation chart in that chapter is rebuilt here, step by step.
- **Working with `nflverse` Data in Python:** `pandas`, and loading `nflverse` data with `nflreadpy`.

The data this chapter uses

Part 13 shows its code, so read the cells as part of the text. Two kinds of data appear.

Season summaries from Next Gen Stats, via `nflverse`. The NFL publishes some tracking-derived numbers for every season, such as each receiver’s average separation and each quarterback’s average time to throw. `nflverse` mirrors them, and `nfl().load_nextgen_stats()` loads them. These are real.

Frame-by-frame tracking, simulated. The raw files, with every player’s position ten times a second, are released only through the **Big Data Bowl** on Kaggle. You need a free account and must accept the competition’s rules to download them, and those rules limit how the data may be shared, so this book does not ship them.¹ Instead, every frame-level figure here uses plays simulated by `gridiron` and written out **in the exact format of the Big Data Bowl files** (same columns, same coordinate system, same quirks), labelled *simulated*, *BDB-format* wherever they appear. That matters more than it sounds: every function in this chapter runs unchanged on the real files. Wherever a figure is simulated, a cell marked “not run here” follows it with the code for the real data, which you can run once you have downloaded a Big Data Bowl release into `data/bdb/`.

```
import tempfile
from pathlib import Path
```

¹Big Data Bowl data is distributed through Kaggle under each competition’s rules, which require accepting the rules before download and restrict use of the data; see the “Rules” and “Data” tabs of, for example, <https://www.kaggle.com/competitions/nfl-big-data-bowl-2025>.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import FancyArrowPatch, Polygon
from IPython.display import Markdown, display

from gridiron import (Play, Field, offense, defense, routes as r, bdb, style, show_animation,
                      frame_strip, draw_frame, draw_marker, FIELD_WIDTH)
from gridiron.data import nfl

style.apply()
FPS = 10 # NFL tracking: 10 frames per second
SIM = Path(tempfile.mkdtemp(prefix="bdb_sim_")) # our stand-in for a Kaggle download folder

def md_table(df, digits=2):
    """Show a small DataFrame as a Markdown table (presentation plumbing, not football)."""
    out = df.copy()
    for c in out.select_dtypes("number"):
        out[c] = out[c].map(lambda v: "" if pd.isna(v) else f"{v:.{digits}f}".rstrip("0").rstrip("."))
    align = ["---:" if pd.api.types.is_numeric_dtype(df[c]) else ":-:" for c in df]
    rows = [list(out.columns), align] + out.astype(str).values.tolist()
    return Markdown("\n".join("| " + " | ".join(map(str, r_)) + " |" for r_ in rows))

```

Where tracking data comes from

Every NFL player wears two small radio chips, one in each shoulder pad. Twenty to thirty receivers mounted around the stadium listen for them, and from the timing of the signals a computer works out where each chip is, about ten times a second. Two chips per player give you not just *where* he is but *which way his shoulders face*. The ball can carry a chip too. The league calls the whole system, and the group of analysts that turns its output into statistics, **Next Gen Stats**.²

Tracking data is the result: the recorded position of every player, many times a second, with speed, acceleration and body angles derived for each moment. **Next Gen Stats** (also called **NGS**) is both the NFL's tracking system and the league group that publishes stats from it.

Why chips, when basketball and baseball use cameras? The NBA had camera tracking in every arena from the 2013-14 season, and baseball's Statcast followed in 2015.³ Football is the hard case for cameras: twenty-two large bodies pile on top of each other at the line, and a camera cannot see a guard who is buried under two defensive tackles. A radio chip does not care what is on top of it. The NFL ran a trial in seventeen stadiums in 2014 and went league-wide the next season; the data has been the backbone of the league's analytics ever since.

What tracking data does *not* record matters as much as what it does:

- **No hands, eyes or feet.** Each player is one point (roughly the middle of his shoulder pads). A tracking file cannot tell you that a corner jammed a receiver, that a guard's hands were outside the frame, or where the quarterback was looking.
- **No assignments.** It records where the free safety *went*, not where he was *supposed* to go. Every "coverage" label you will meet in a tracking file was added by people watching film (PFF, in the Big Data Bowl files) or by a model trained on such labels.
- **A little noise.** Positions are accurate to within a few inches, not to the millimetre, and a player standing still seems to drift slightly from frame to frame. That is why speeds near zero are unreliable and why careful analysts smooth the data before taking derivatives.

²NFL Football Operations, "NFL Next Gen Stats" (RFID tags in each player's shoulder pads, 20-30 receivers in each stadium, location, speed and acceleration about 10 times a second): <https://operations.nfl.com/gameday/technology/nfl-next-gen-stats/>; on the 2014 trial in 17 stadiums, Kevin Seifert, ESPN, "Inside Slant: NFL Next Gen Stats opens new doors" (Dec. 11, 2014): https://www.espn.com/blog/nflnation/post/_/id/153569/inside-slant-nfl-next-gen-stats-opens-new-doors.

³NBA: STATS LLC's SportVU camera system was installed in every arena for the 2013-14 season; MLB: Statcast was installed in every park for the 2015 season. See, for example, <https://www.nba.com/news/nba-stats-sportvu-player-tracking> and <https://www.mlb.com/glossary/statcast>.

🔥 Common misconception

“Tracking data knows the coverage.” It knows where eleven defenders stood and moved. Whether that was Cover 3 or quarters is an *interpretation*, and two careful charting services can disagree about it. The man/zone and coverage columns in the Big Data Bowl files are charted labels sitting next to the tracking, not part of it. Much of the best Big Data Bowl work is about teaching a model to make that interpretation from the dots.

What is public, and when

Three layers of tracking data reach the public, at different speeds:

What	Detail	When you get it
Broadcast graphics	One number on one play (speed, separation, completion probability)	Live
NGS season and weekly summaries	Per-player averages: separation, cushion, time to throw, expected rushing yards and more; on nextgenstats.nfl.com and in nflverse	During the season, weekly
Raw frames (x, y, speed, angles for all 22 players)	Big Data Bowl releases only: a slice of one or two past seasons	Once a year, for the competition

So the raw frames you can work with always lag the game you watched by a season or more. The current release covers the 2023 season (the 2026 Big Data Bowl); the release with the richest pre-snap data covers the first nine weeks of 2022 (the 2025 Big Data Bowl). Everything in this chapter is written to work on both.

One row, one player, one tenth of a second

A tracking file is a long table. Each row is **one player in one frame**. A **frame** is one snapshot of everyone on the field at one instant; at ten frames a second, a five-second play produces about fifty frames, so with 22 players plus the ball a single play is over a thousand rows, and one week of games is millions.

The field as a grid

Before the columns make sense, you need the grid they live on. It is the same grid every diagram in this book has been drawn on since the first chapter.

```
fig, ax = plt.subplots(figsize=(6.5, 3.75))
fld = Field(ax=ax, orient="horizontal", window=None, to_go=None)
ink = dict(color=style.INK, fontsize=7.5)
grey = dict(color=style.INK_2, fontsize=7)
arrow = dict(arrowstyle="->", mutation_scale=9, color=style.INK, lw=1.0)

# axes drawn just outside the field
ax.add_patch(FancyArrowPatch((0, -2.5), (123, -2.5), **arrow))
ax.add_patch(FancyArrowPatch((-2.5, 0), (-2.5, 56.5), **arrow))
TICKS = ((0, "0"), (10, "10\ngoal line"), (60, "60\nmidfield"), (110, "110\ngoal line"), (120, "120"))
for xv, txt in TICKS:
    ax.plot([xv, xv], [-2.0, -3.0], color=style.INK, lw=0.8)
    ax.text(xv, -4.0, txt, ha="center", va="top", **grey)
ax.text(123.5, -2.5, "x (yd)", ha="left", va="center", **ink)
for yv, txt in ((0, "0"), (FIELD_WIDTH, "53.3")):
    ax.text(-3.5, yv, txt, ha="right", va="center", **grey)
ax.text(-2.5, 57.5, "y (yd)", ha="center", va="bottom", **ink)
ax.plot(0, 0, "o", color=style.INK, ms=4, zorder=12)
ax.text(1.5, 1.5, "(0, 0)", ha="left", va="bottom", **ink)
```

```

# one player and his coordinates
draw_marker(ax, 35, 20, "off", "", 2.0, r=1.0)
ax.plot([35, 35], [0, 18.8], color=style.INK_2, lw=0.8, ls=(0, (2, 2)), zorder=5)
ax.plot([0, 33.8], [20, 20], color=style.INK_2, lw=0.8, ls=(0, (2, 2)), zorder=5)
ax.text(37, 17.0, "x = 35, y = 20", ha="left", va="center", zorder=12, **ink)

# hash labels and the direction of play after standardising
for yh in (23.58, 29.75):
    ax.text(3.0, yh, f"hash {yh:.1f}", ha="left", va="center", **grey)
ax.add_patch(FancyArrowPatch((30, 47.0), (64, 47.0), arrowstyle="-|>", mutation_scale=12,
                             color=style.OFFENSE_DARK, lw=2.0, zorder=6))
ax.text(47, 49.5, "offense moves toward +x (after standardising)", ha="center", va="bottom",
        color=style.OFFENSE_DARK, fontsize=7.5)

# the angle convention: clockwise from +y
cx, cy, rad = 86.0, 27.0, 6.0
for deg, lab in ((0, "0°"), (90, "90°"), (180, "180°"), (270, "270°")):
    dx, dy = np.sin(np.radians(deg)), np.cos(np.radians(deg))
    ax.add_patch(FancyArrowPatch((cx, cy), (cx + rad * dx, cy + rad * dy), arrowstyle="-|>",
                                mutation_scale=8, color=style.THIRD, lw=1.4, zorder=6))
    ax.text(cx + (rad + 2.6) * dx, cy + (rad + 2.0) * dy, lab, ha="center", va="center",
            color=style.INK, fontsize=7, zorder=12)
ax.set_xlim(-9, 131)
ax.set_ylim(-11, 60)
plt.show()

```

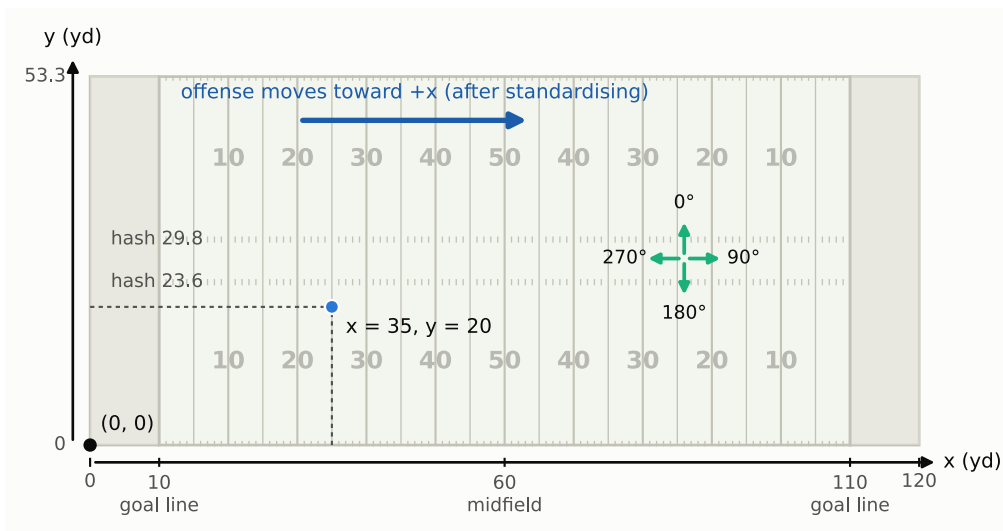


Figure 1: The tracking coordinate system. x runs the length of the field, 0 to 120 yards, so the goal lines sit at $x = 10$ and $x = 110$ and midfield at $x = 60$. y runs across it, 0 to 53.3 yards; the hash marks sit at about $y = 23.6$ and $y = 29.8$. The point $(0, 0)$ is a corner of the field at the back of one end zone. The player shown stands at $x = 35$, $y = 20$: his own 25-yard line if his team is going to the right, nearer the $y = 0$ sideline. Angles (orientation and direction) are measured in degrees clockwise from the $+y$ direction, so 90 degrees points toward $x = 120$ and 270 degrees toward $x = 0$. After standardising (next section), every offense moves toward $+x$.

Three facts from that picture will save you hours:

- **The origin is a corner, not the ball.** Yard line numbers on the field ("the 25") are not x values. The 25-yard line nearest $x = 0$ is $x = 35$; the one nearest $x = 120$ is $x = 85$. Everything you compute about the line of scrimmage starts by finding the ball's x at the snap.

- **The field is 160 feet wide, which is 53½ yards.** The files round it to 53.3. That gap of a few centimetres never matters for football, but it does matter for code: use one constant everywhere, or a play flipped twice will not land where it started. `gridiron` uses the exact 160/3.
- **Angles are compass-like, but not math-like.** School trigonometry measures angles counter-clockwise from the x axis. Tracking angles run *clockwise* from the +y direction. To turn an angle into a direction vector, use `dx = sin(angle)`, `dy = cos(angle)`, not the other way round. Get this wrong and every receiver appears to run sideways.

The columns

Now the file itself. Here is the simulated play this chapter follows from start to finish, the **chapter play**: an 11-personnel 2x2 set (one back, one tight end, three receivers, two on each side), with the slot receiver H sent in motion across the formation, against a nickel defense that shows two-high safeties and rotates at the snap into a Cover 1 robber look: one safety spins to the deep middle, the other comes down into the middle of the field as a robber, the corners, the nickel and the Will linebacker play man, and the Mike sits in the middle watching the back. The quarterback throws a comeback to Z, the receiver wide to the right, on time: the ball leaves his hand as Z starts his break.

```
def chapter_play(target="Z", t_throw=1.95, los=45):
    """The chapter play: 11 personnel 2x2 vs a nickel two-high shell that rotates to Cover 1 robber."""
    off = offense("gun_2x2")
    dfn = defense("nickel", "two_high", off)
    spot = {"WILL": (5.0, -6.0), "LCB": (7.0, -15.8), "RCB": (7.0, 15.8),
            "FS": (13.0, -7.5), "SS": (13.0, 7.5)}
    dfn = [q.moved(d=spot[q.key][0], w=spot[q.key][1]) if q.key in spot else q for q in dfn]
    p = Play(off, dfn, los=los, to_go=10, title="The chapter play")
    # before the snap: the safeties bounce in place, then H motions across the formation
    p.motion("FS", to=(13.3, -7.3), speed=0.1)
    p.motion("SS", to=(12.8, 7.4), speed=0.07)
    p.motion("H", to=(-2.2, 3.5), speed=6.0)          # still moving at the snap
    # protection: tackles kick back, the line gives ground late; four rushers
    for k, pts in {"LT": [(-1.6, -1.0), (-3.2, -0.5), (-4.6, 0.6)], "LG": [(-1.2, -0.3), (-2.4, 0.2)],
                  "C": [(-1.0, 0.0), (-2.0, 0.0)], "RG": [(-1.2, 0.3), (-2.4, -0.2)],
                  "RT": [(-1.6, 1.0), (-3.2, 0.5), (-4.4, -0.5)]}.items():
        p.block(k, pts=pts)
    p.block("RB", pts=[(0.3, -1.0)])                  # the back sets up beside the quarterback
    for k, pts in {"WDE": [(-2.0, -1.8), (-4.8, -0.8), (-7.6, 2.2)],
                  "WDT": [(-1.6, 0.0), (-3.0, 0.3)], "SDT": [(-1.5, 0.2), (-3.0, -0.2)],
                  "SDE": [(-2.0, 1.8), (-4.6, 0.9), (-7.4, -1.8)]}.items():
        p.rush(k, pts=pts, speed=4.4)
    p.dropback(2.5)
    p.route("X", r.go(24))
    p.route("Z", r.path((14, 0), (11.5, 2.5), (8.5, 2.0))) # comeback, working back toward the ball
    p.route("Y", r.seam(18)); p.route("H", r.arrow(3, 6))
    for d_, o_ in (("LCB", "X"), ("RCB", "Z"), ("NB", "Y"), ("WILL", "H")):
        p.man(d_, o_)                                # man coverage; the Will travels with H
    p.drop("MIKE", (5.5, 0.0), speed=2.0)            # the Mike sits in the middle, eyes on the back
    p.drop("FS", (16.5, 0.0), speed=6.5)             # spins to the deep middle at the snap
    p.drop("SS", (9.0, 2.0), speed=5.5)              # comes down as the robber
    p.pass_to(target, t=t_throw)
    return p

play = chapter_play()
play.draw(legend=True, width_in=6.2, lateral=(-19, 19))
plt.show()
```

The chapter play

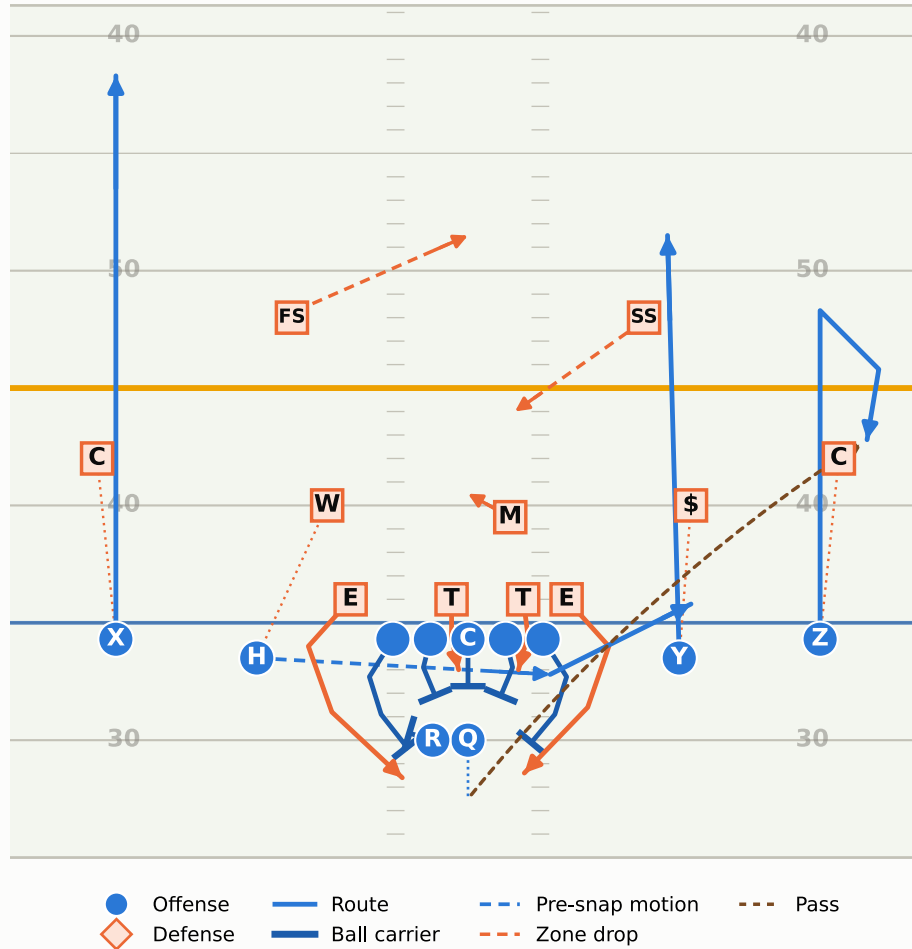


Figure 2: The chapter play as a playbook diagram (simulated). Offense: H's dashed line is his motion across the formation before the snap; X runs a go, Y a seam, Z a comeback and H a short arrow route to the flat; the back stays in to block; the dashed brown arrow is the throw to Z. Defense: the dotted lines tie each man defender to his receiver (corners on X and Z, the nickel on Y, the Will on H); the free safety spins from the left hash to the deep middle and the strong safety comes down as the robber; the Mike drifts into the middle; four rush.

A `gridiron` play can write itself out as tracking rows. The function below does that, then makes the rows look like a real Big Data Bowl 2025 file: it adds the sensor wobble real chips have, adds which way each player is *facing* (the simulator moves dots but does not turn shoulders), turns the play around so that this offense is going toward the **left** end zone, as half of all real plays do, and writes the three files a real download would contain: `tracking_week_1.csv`, `plays.csv` and `players.csv`.

```
JERSEY = {"QB": 7, "RB": 22, "X": 11, "H": 84, "Y": 88, "Z": 17, "LT": 71, "LG": 65, "C": 60,
          "RG": 64, "RT": 76, "WDE": 91, "WDT": 97, "SDT": 95, "SDE": 99, "MIKE": 54, "WILL": 50,
          "LCB": 24, "RCB": 21, "NB": 29, "FS": 31, "SS": 33}
POSITION = {"QB": "QB", "RB": "RB", "X": "WR", "Z": "WR", "H": "WR", "Y": "TE", "LT": "T", "RT": "T",
            "LG": "G", "RG": "G", "C": "C", "WDE": "DE", "SDE": "DE", "WDT": "DT", "SDT": "DT",
            "MIKE": "ILB", "WILL": "ILB", "LCB": "CB", "RCB": "CB", "NB": "CB", "FS": "FS", "SS": "SS"}
COLUMNS = ["gameId", "playId", "nflId", "displayName", "frameId", "frameType", "time", "jerseyNumber",
            "club", "playDirection", "x", "y", "s", "a", "dis", "o", "dir", "event"]
```

```
def bdb_angle(vx, vy):
    """Tracking angle of a velocity: degrees clockwise from +y (0 = toward +y, 90 = toward +x)."""
    return (90 - np.degrees(np.arctan2(vy, vx))) % 360
```

```

def as_bdb_rows(p, game_id, play_id, direction="left", seed=0, wobble=0.015):
    """A gridiron Play as rows of a BDB 2025 tracking_week_N.csv (camelCase columns)."""
    rng = np.random.default_rng(seed)
    t = p.to_tracking(fps=FPS, game_id=game_id, play_id=play_id)
    passes = [(to, te) for kind, to, te in p.ball_events if kind == "pass"]
    out = []
    for key, g in t.groupby("displayName", sort=False):
        g = g.sort_values("frameId").copy()
        if key != "football":
            g["x"] += rng.normal(0, wobble, len(g)) # chips wobble a little, even at rest
            g["y"] += rng.normal(0, wobble, len(g))
            # speed and direction from the last few tenths of a second (backward differences,
            # averaged over three frames, as tracking providers smooth theirs)
            dx, dy = np.diff(g["x"].to_numpy()), np.diff(g["y"].to_numpy())
            smooth = lambda v: pd.Series(v).rolling(3, min_periods=1).mean().to_numpy() # noqa: E731
            vx, vy = smooth(np.r_[dx[0], dx] * FPS), smooth(np.r_[dy[0], dy] * FPS)
            g["s"] = np.hypot(vx, vy)
            g["a"] = np.abs(np.r_[0.0, np.diff(g["s"].to_numpy())] * FPS)
            g["dis"] = np.r_[0.0, np.hypot(dx, dy)]
            g["dir"] = bdb_angle(vx, vy)
            facing = 90.0 if g["side"].iloc[0] == "off" else 270.0 # toward the other team
            o = np.where(g["s"] > 1.5, g["dir"], facing) # running: face where you go
            if key == "QB" and passes:
                o = np.where(g["time"] <= passes[0][1], 90.0, o) # eyes downfield until he throws
            if POSITION[key] in ("CB", "FS", "SS", "ILB"):
                o = np.where(g["time"] < 0.9, 270.0, o) # backpedal facing the QB
            g["o"] = (o + rng.normal(0, 3, len(g))) % 360
        out.append(g)
    t = pd.concat(out)
    # events a real file carries: line set, motion, snap, throw, arrival, catch
    ev = t.set_index(["displayName", "frameId"])["event"].copy()
    mot = [a for a in p.actions if a.kind == "motion" and p.players[a.key].side == "off"]
    if mot:
        f_mot = int(round(min(a.t0 for a in mot) * FPS)) + int(-t["time"].min() * FPS + 1.5)
        ev.loc[("football", f_mot)] = "man_in_motion"
        ev.loc[("football", max(1, f_mot - 6))] = "line_set"
    if passes:
        to, te = passes[0]
        ball = t[t["displayName"] == "football"].set_index("frameId")
        rec = t[t["displayName"] == to].set_index("frameId")
        gap = np.hypot(ball["x"] - rec["x"], ball["y"] - rec["y"])
        f_arr = int(gap[(ball["time"] > te) & (gap < 0.6)].index.min())
        ev.loc[("football", f_arr)] = "pass_arrived"
        ev.loc[("football", f_arr + 1)] = "pass_outcome_caught"
    t["event"] = ev.reindex(pd.MultiIndex.from_frame(t[["displayName", "frameId"]])).to_numpy()
    # a real file stores events on every player's row for that frame
    first_event = lambda s: s.dropna().iloc[0] if s.notna().any() else None # noqa: E731
    t["event"] = t.groupby("frameId")["event"].transform(first_event)
    if direction == "left":
        # turn the play around: heading to x = 0
        t["x"], t["y"] = 120 - t["x"], FIELD_WIDTH - t["y"]
        t["o"], t["dir"] = (t["o"] + 180) % 360, (t["dir"] + 180) % 360
    t["playDirection"] = direction
    t["jerseyNumber"] = t["displayName"].map(JERSEY)

```

```

t0 = pd.Timestamp("2026-09-13 13:02:11")
stamp = t0 + pd.to_timedelta((t["frameId"] - 1) / FPS, unit="s")
t["time"] = stamp.dt.strftime("%Y-%m-%d %H:%M:%S.%f")
for c in ("x", "y", "s", "a", "dis", "o", "dir"):
    t[c] = t[c].round(2)
return t.sort_values(["frameId", "nflId"], na_position="last")[COLUMNS]

def write_week(folder, games):
    """Write tracking_week_1.csv, plays.csv and players.csv for (play_id, Play, direction, meta)."""
    rows, plays, people = [], [], {}
    for play_id, p, direction, meta in games:
        rows.append(as_bdb_rows(p, 2026091300, play_id, direction, seed=play_id))
        plays.append({"gameId": 2026091300, "playId": play_id, "possessionTeam": "OFF",
                     "defensiveTeam": "DEF", **meta})
        for i, key in enumerate(p.players):
            people[1000 + i] = (key, POSITION[key])
    pd.concat(rows).to_csv(folder / "tracking_week_1.csv", index=False)
    pd.DataFrame(plays).to_csv(folder / "plays.csv", index=False)
    roster = pd.DataFrame([(k, n, pos) for k, (n, pos) in people.items()],
                          columns=["nflId", "displayName", "position"])
    roster.to_csv(folder / "players.csv", index=False)

write_week(SIM, [(101, play, "left", {"playDescription": "(simulated) QB pass short right to Z",
                                     "pff_passCoverage": "Cover-1", "pff_manZone": "Man"})])
print(sorted(f.name for f in SIM.iterdir()))

```

```
['players.csv', 'plays.csv', 'tracking_week_1.csv']
```

Now load it exactly as you would load a real week, and look at one player's first rows.

```

raw = pd.read_csv(SIM / "tracking_week_1.csv")
players = pd.read_csv(SIM / "players.csv")
cb = raw[raw["displayName"] == "LCB"]
f_snap = int(cb.loc[cb["frameType"] == "SNAP", "frameId"].iloc[0])
show_frames = [1, 2, 3, f_snap, f_snap + 6, f_snap + 7, f_snap + 8]
print("club:", cb["club"].iloc[0], "| playDirection:", cb["playDirection"].iloc[0])
cols = ["frameId", "frameType", "x", "y", "s", "a", "dis", "o", "dir", "event"]
md_table(cb[cb["frameId"].isin(show_frames)][cols].fillna(""))

```

```
club: DEF | playDirection: left
```

frameId	frame- Type	x	y	s	a	dis	o	dir	event
1	BEFORE_SNAP	67.99	10.88	0.21	0	0	87.53	184.31	
2	BEFORE_SNAP	67.99	10.86	0.21	0	0.02	91.24	184.31	
3	BEFORE_SNAP	68	10.86	0.14	0.64	0.01	86.16	165.54	
38	SNAP	68.03	10.84	0.1	0.34	0.03	90.99	89.22	ball_snap
44	AFTER_SNAP	68.7	11.15	1.87	11.53	0.34	86.65	284.33	

frameId	frame- Type	x	y	s	a	dis	o	dir	event
45	AFTER_SNAP	68.39	11.2	3.16	12.9	0.32	92.49	280.33	
46	AFTER_SNAP	68.02	11.22	3.4	2.48	0.37	88.94	278.2	

Read the columns from left to right:

Column	Meaning	Units
<code>gameId</code> , <code>playId</code>	Which game and which play. Together they identify a play everywhere in the release.	
<code>nflId</code> , <code>displayName</code> , <code>jerseyNumber</code> , <code>club</code>	Who the row is about. The ball has its own rows, with <code>displayName</code> “football” and no <code>nflId</code> .	
<code>frameId</code>	The frame number, counting from 1 at the start of the play’s recording.	0.1 s per frame
<code>frameType</code>	<code>BEFORE_SNAP</code> , <code>SNAP</code> or <code>AFTER_SNAP</code> (Big Data Bowl 2025 files).	
<code>time</code>	The wall-clock timestamp of the frame.	
<code>playDirection</code>	Which end zone the offense was heading for: <code>right</code> (toward <code>x = 120</code>) or <code>left</code> .	
<code>x</code> , <code>y</code>	Position on the grid above.	yards
<code>s</code>	Speed.	yards per second
<code>a</code>	Acceleration (its size, not its direction).	yards per second ²
<code>dis</code>	Distance travelled since the previous frame.	yards
<code>o</code>	Orientation: the way the player’s shoulders face.	degrees
<code>dir</code>	Direction: the way he is moving.	degrees
<code>event</code>	What happened on that frame, when anything did: <code>line_set</code> , <code>man_in_motion</code> , <code>ball_snap</code> , <code>pass_forward</code> , <code>pass_arrived</code> , <code>pass_outcome_caught</code> , <code>tackle</code> and so on.	

Speeds are in yards per second, which few people have a feel for. Multiply by about **2.05** to get miles per hour: a receiver at full speed runs about 9 to 10 yards a second (18 to 21 mph), and the fastest ball carriers each season touch 11 or more (22-plus mph, the numbers broadcasts love). A quarterback’s dropback is 3 to 4 yards a second; a corner’s backpedal about the same.

Notice two things in the corner’s rows (he is the left cornerback, and on this play the offense is going toward `x = 0`). First, in the early, pre-snap frames he is standing still, yet `s` is not exactly zero and `dir` jumps around: that is the sensor wobble. When a player is not moving, his direction is meaningless, and any feature built on `dir` should ignore frames with `s` below about half a yard a second. Second, look at the last three rows, six to eight frames after the snap: he is moving at about 3 yards a second with `dir` near 280 (toward `x = 0`, away from the line: he is backpedalling), while his `o` stays near 90 (he is facing the offense). Direction and orientation point almost opposite ways. That is the most important subtlety in the file.

Orientation versus direction

Orientation vs direction is the difference between where a player *faces* (o) and where he *goes* (dir). Most of the time they agree: a receiver sprinting downfield faces downfield. But football is full of players moving one way while looking another, and those are exactly the players you want to understand.

```
trk = bdb.load_tracking(str(SIM / "tracking_week_*.csv"))      # the same call works on real files
plays_csv = pd.read_csv(SIM / "plays.csv")
one = bdb.prepare(trk, plays_csv, game_id=2026091300, play_id=101) # offense now moves toward +x
LABEL = {k: q.label for k, q in play.players.items()}
one["label"] = one["displayName"].map(LABEL).fillna("")
snap_ball = one[(one["frameType"] == "SNAP") & (one["side"] == "ball")].iloc[0]
LOS, BALL_Y = float(snap_ball["x"]), float(snap_ball["y"])
one["d"], one["w"] = one["x"] - LOS, BALL_Y - one["y"]          # the book's diagram frame
one["frame_rel"] = np.round(one["time"] * FPS).astype(int)      # frames from the snap
one["_id"] = one["displayName"]                                 # draw_frame() groups trails by this
EVENT_T = one.dropna(subset=["event"]).drop_duplicates("event").set_index("event")["time"]
T_THROW, T_ARR = float(EVENT_T["pass_forward"]), float(EVENT_T["pass_arrived"])

def arrow_vec(theta, length):
    """A tracking angle as a vector in the vertical (offense-up) diagram view."""
    rad = np.radians(theta)
    return -np.cos(rad) * length, np.sin(rad) * length

fig, ax = plt.subplots(figsize=(6.5, 4.5))
fld = Field(ax=ax, orient="vertical", los=LOS, window=(-13, 19), to_go=10,
            lateral=(BALL_Y - 24, BALL_Y + 24))
now = one[one["frame_rel"] == 6]
draw_frame(fld, now, one[one["frame_rel"].between(0, 6)])
notes = {"QB": ("QB: faces downfield (o ≈ 90°),\nmoves straight back (dir ≈ 270°)", (-10.5, -11.0)),
         "FS": ("FS: faces the QB (o ≈ 270°),\nruns to the middle (dir ≈ 150°)", (12.0, -14.0)),
         "WDE": ("Left end: faces and moves\nthe same way (o ≈ dir)", (-4.0, -16.0))}
for key, (txt, (d_, w_)) in notes.items():
    row = now[now["displayName"] == key].iloc[0]
    px, py = fld.xy(row["x"], row["y"])
    for ang, col, ls in ((row["o"], style.INK, "-"), (row["dir"], style.THIRD, (0, (2, 1.5)))):
        dx, dy = arrow_vec(ang, 3.2)
        ax.add_patch(FancyArrowPatch((px, py), (px + dx, py + dy), arrowstyle="-|>", mutation_scale=9,
                                     color=col, lw=1.8, ls=ls, zorder=13))
    tx, ty = fld.xy(LOS + d_, BALL_Y - w_)
    ax.text(tx, ty, txt, ha="center", va="center", fontsize=7, color=style.INK, zorder=14,
            bbox=dict(boxstyle="round,pad=0.25", fc=style.SURFACE, ec=style.BASELINE, lw=0.6))
plt.show()
```

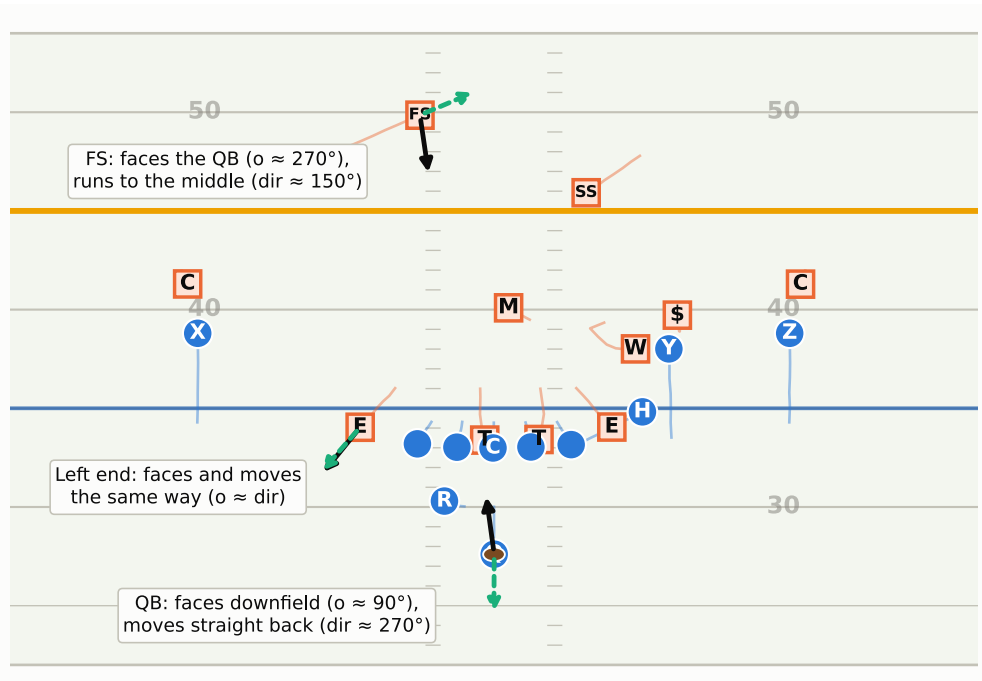


Figure 3: Orientation (solid black arrow, where the shoulders face) against direction (dashed green arrow, where the player is moving), 0.6 seconds after the snap of the chapter play (simulated, BDB-format). The quarterback faces downfield while dropping straight back: the two angles are opposite. The free safety faces the quarterback while running sideways and deeper to the middle of the field, his rotation: the angles differ by more than 90 degrees. The left defensive end, rushing, faces and moves the same way. A model that used only direction would think the quarterback was running away from the play and the safety was turning his back to it.

Where does this matter? Everywhere a defender reads the play. A corner in off coverage backpedals facing the quarterback; the instant he *turns* (his *o* swings toward his *dir*) he has committed, and a receiver who breaks at that moment wins. A zone defender keeps facing the quarterback as he drops; a man defender turns to face his receiver. A pass rusher who has been turned sideways by a blocker is moving toward the quarterback but facing the sideline. Several Big Data Bowl entries have built their best features from the gap between the two angles.

Standardising play direction

Half of all plays go left. If you average where the free safety lines up on a thousand plays without fixing that, half of him is at $x = 70$ and half at $x = 50$, and the average puts him on the ball. The fix is the first thing every tracking analysis does.

Standardised play direction means rotating every play that went toward the left end zone by 180 degrees, so that all plays move left to right. A rotation needs **both** coordinates flipped and both angles turned around:

```
def standardise(df):
    """Rotate plays with playDirection == 'left' by 180 degrees, so every offense moves toward +x."""
    df = df.copy()
    left = df["playDirection"] == "left"
    df.loc[left, "x"] = 120 - df.loc[left, "x"]
    df.loc[left, "y"] = FIELD_WIDTH - df.loc[left, "y"] # flip y too: a rotation, not a mirror
    for col in ("o", "dir"):
        df.loc[left, col] = (df.loc[left, col] + 180) % 360
    df.loc[left, "playDirection"] = "right"
    return df

mine = standardise(trk)
theirs = bdb.normalize_direction(trk) # gridiron's version of the same function
```

```
print("largest difference from gridiron's version:",
      float((mine[["x", "y"]] - theirs[["x", "y"]]).abs().max().max()), "yards")
```

```
largest difference from gridiron's version: 0.0 yards
```

Why flip y as well as x? Picture yourself standing behind the offense. If you flip only x, the offense now faces the other way but the receiver who was on the quarterback's left is still at the same y, and from the quarterback's new point of view that is his *right*. You have made a mirror image, and every left-handed fact about the play (which hash, which side the tight end is on, the boundary) is now wrong. Flipping both is a half-turn of the whole field: nothing changes sides.

```
KEYS = ["X", "Z", "QB", "FS", "SS"]
snap_raw = trk[(trk["frameType"] == "SNAP")].copy()
views = {
    "(a) as recorded: playDirection = left": snap_raw,
    "(b) x flipped only: a mirror image": snap_raw.assign(x=120 - snap_raw["x"]),
    "(c) x and y flipped: standardised": standardise(snap_raw),
}
fig, axes = plt.subplots(2, 2, figsize=(6.5, 3.9))
for ax, (name, df) in zip(axes.flat[:3], views.items()):
    Field(ax=ax, orient="horizontal", window=None, numbers=False)
    for key in KEYS + ["football"]:
        row = df[df["displayName"] == key].iloc[0]
        side = {"OFF": "off", "DEF": "def"}.get(row["club"], "ball")
        if side == "ball":
            ax.plot(row["x"], row["y"], "o", color=style.BALL, ms=3, zorder=11)
            continue
        draw_marker(ax, row["x"], row["y"], side, "", 1.0, r=2.2)
        ax.text(row["x"], row["y"] + (4.2 if key != "Z" else -4.2), key, ha="center",
                va="bottom" if key != "Z" else "top", fontsize=6.5, color=style.INK, zorder=12)
    ball = df[df["displayName"] == "football"].iloc[0]
    heading = -1 if "(a)" in name else 1 # which way this offense is moving
    tail, tip = (ball["x"] + heading * 2, ball["y"]), (ball["x"] + heading * 8, ball["y"])
    ax.add_patch(FancyArrowPatch(tail, tip, arrowstyle="-|>", mutation_scale=8,
                                color=style.OFFENSE_DARK, lw=1.4, zorder=6))
    ax.set_title(name, fontsize=7.5, loc="left", color=style.INK)
ax = axes.flat[3]
Field(ax=ax, orient="vertical", los=45, window=(-8, 16), lateral=(0, FIELD_WIDTH), numbers=False)
st = standardise(snap_raw)
for key in KEYS:
    row = st[st["displayName"] == key].iloc[0]
    side = "off" if row["club"] == "OFF" else "def"
    px, py = FIELD_WIDTH - row["y"], row["x"]
    draw_marker(ax, px, py, side, "", 1.0, r=1.4)
    ax.text(px + (2.2 if key != "Z" else -2.2), py, key, ha="left" if key != "Z" else "right",
            va="center", fontsize=6.5, color=style.INK, zorder=12)
ax.set_title("(d) the diagram frame: d = x - LOS, w = ball y - y", fontsize=7.5, loc="left")
fig.tight_layout()
plt.show()
```

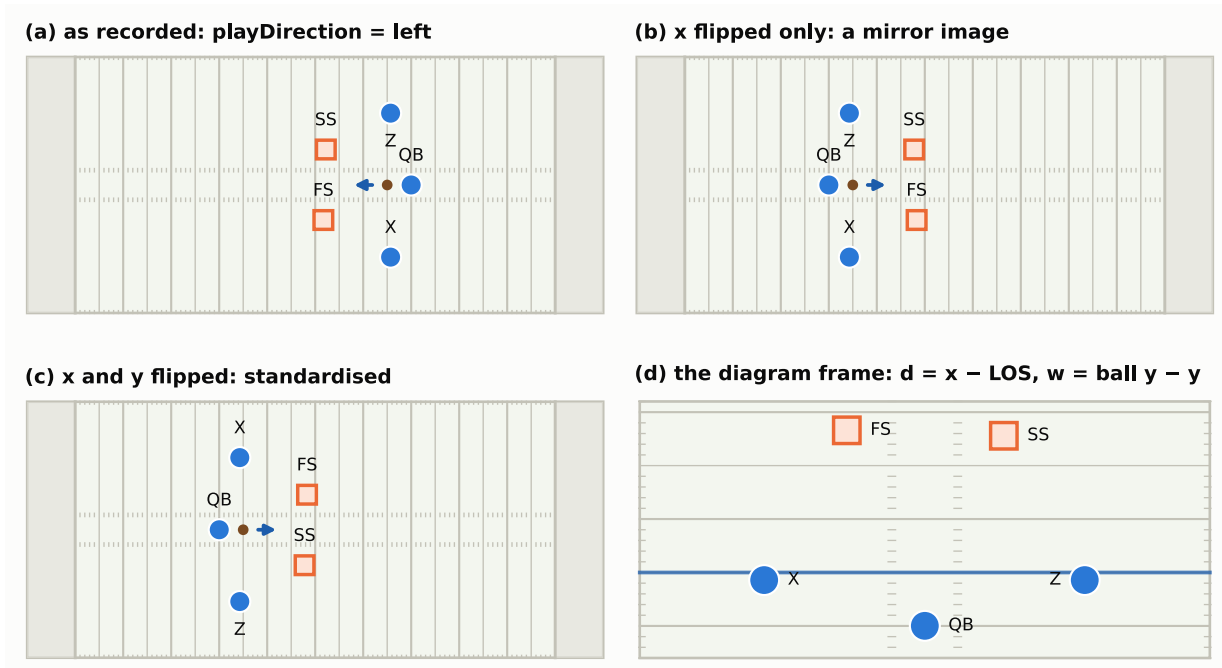


Figure 4: Standardising one play (simulated, BDB-format), shown at the snap with five players: the two outside receivers X and Z, the quarterback and the two safeties. (a) As recorded, the offense is heading for the left end zone ($x = 0$); X, the receiver on the quarterback's left, is on the low-y side. (b) Flipping only x turns the offense around but mirrors the play: X is now on the quarterback's right. (c) Flipping x and y rotates the play: the offense heads right and X is still on its left. (d) The same snap in the book's diagram frame, d yards downfield and w yards to the offense's right of the ball, drawn offense-up: this is the frame every play diagram in the book uses.

Panel (d) is worth a second look. Every diagram in this book is authored in two numbers: **d**, yards downfield of the line of scrimmage, and **w**, yards to the offense's right of the ball. After standardising, those are just $x - \text{LOS}$ and $\text{ball}_y - y$ at the snap. That is not a coincidence: the library was built so that a play a coach draws and a play the chips record are the same kind of object, and one drawing routine serves both.

! Why it exists

Every question worth asking of tracking data is a comparison: this safety against that one, this week against last, this team against the league. Comparisons need a common frame, and the raw files don't give you one: the same coverage looks like two opposite pictures depending on which end zone the offense faced. Standardising costs nothing and loses nothing (it is reversible), but forget it once and every average in the analysis is meaningless.

Lining plays up: the snap and the throw

The second alignment problem is time. Frame 1 is wherever recording started, which differs from play to play: one play has three seconds of frames before the snap, another twelve. So the next step is to re-number frames **relative to an event**: `frame_rel = frameId - frameId of the event`. Frame 0 is then the event itself, negative frames come before it, positive after.

The snap is the natural zero for most questions: pre-snap disguise, motion, how fast the rushers get off the ball. But many passing questions are really about **the throw**: how open was the receiver *when the ball left the hand*, how close was the pressure *then*. Quarterbacks throw anywhere from about 1.5 to over 4 seconds after the snap, so a chart aligned on the snap smears the moment you care about across thirty frames. Aligned on the throw, it lines up.

```
rng = np.random.default_rng(5)
choices = {"H": (1.4, 1.9), "Y": (1.9, 2.5), "Z": (2.2, 2.7), "X": (2.6, 3.2)}
rows = []
for pid in range(40):
    target = rng.choice(list(choices))
```

```

p = chapter_play(target=target, t_throw=float(rng.uniform(*choices[target])))
t = p.to_tracking(fps=FPS, play_id=pid)
ball = t[t["displayName"] == "football"].copy()
f_snap = int(ball.loc[ball["event"] == "ball_snap", "frameId"].iloc[0])
f_throw = int(ball.loc[ball["event"] == "pass_forward", "frameId"].iloc[0])
ball["since_snap"] = ball["frameId"] - f_snap
ball["since_throw"] = ball["frameId"] - f_throw
ball["depth"] = ball["x"] - p.los
rows.append(ball.assign(target=target))
balls = pd.concat(rows)

fig, axes = plt.subplots(1, 2, figsize=(6.5, 3.0), sharey=True)
for ax, col, lab in ((axes[0], "since_snap", "Frames since the snap"),
                    (axes[1], "since_throw", "Frames since the throw")):
    for _, g in balls.groupby("playId"):
        g = g[g[col].between(-10, 22) if col == "since_throw" else g[col].between(0, 42)]
        ax.plot(g[col], g["depth"], color=style.SERIES[0], lw=0.8, alpha=0.5)
    ax.axvline(0, color=style.INK_2, lw=1)
    ax.axhline(0, color=style.BASELINE, lw=0.8)
    ax.set_xlabel(lab, fontsize=8.5)
axes[0].set_ylabel("Ball, yards past the line of scrimmage", fontsize=8.5)
axes[0].set_title("Aligned on the snap", fontsize=9.5)
axes[1].set_title("Aligned on the throw", fontsize=9.5)
fig.tight_layout()
plt.show()

```

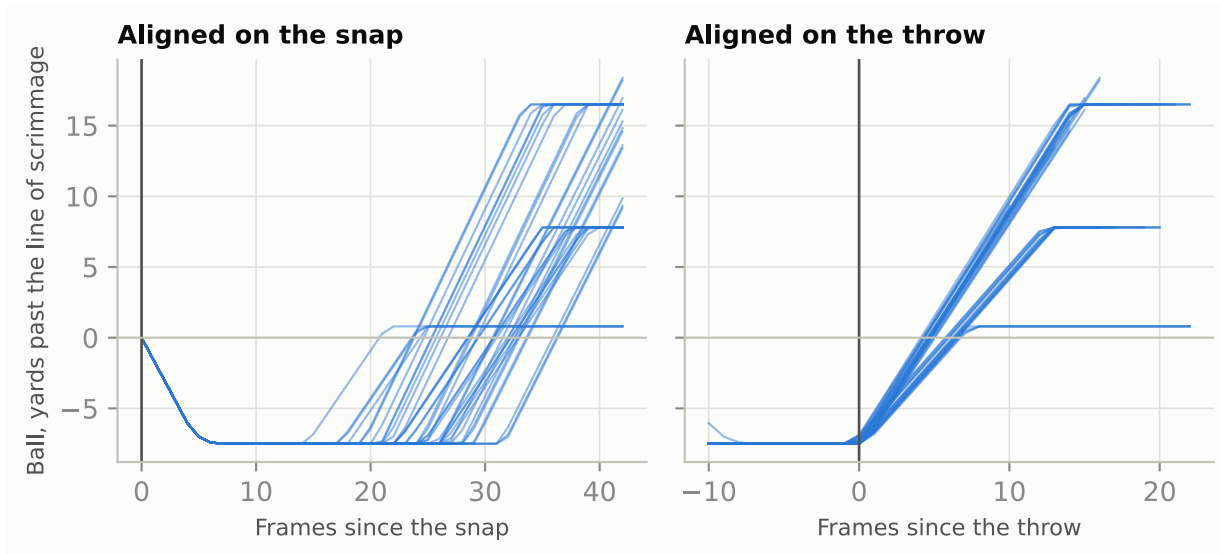


Figure 5: Forty simulated dropbacks (BDB-format), each line one play: how far downfield the ball is. Left, aligned on the snap: the ball goes back with the quarterback, then leaves on a throw anywhere from about 1.5 to 3.2 seconds later, and the flights are spread across the chart. Right, the same plays aligned on the pass_forward event: every flight starts at frame 0, and differences between plays (short throws flatten out quickly, deep ones keep climbing) become visible.

In code, alignment is a merge: find each play's event frame once, join it back, subtract.

```

def align(frames, event="ball_snap"):
    """Add frame_rel: frames since `event` (negative = before it). Plays without the event drop out."""
    ev = (frames.loc[frames["event"] == event, ["gameId", "playId", "frameId"]]
          .drop_duplicates(["gameId", "playId"]).rename(columns={"frameId": "f0"}))
    out = frames.merge(ev, on=["gameId", "playId"])

```

```
out["frame_rel"] = out["frameId"] - out["f0"]
return out.drop(columns="f0")
```

```
at_throw = align(mine, "pass_forward")
print("frames before the throw:", -at_throw["frame_rel"].min(),
      "| after:", at_throw["frame_rel"].max())
```

```
frames before the throw: 57 | after: 22
```

Two practical warnings. **Events are not always where you expect.** The snap event name has varied (`ball_snap`, and `snap_direct` for direct snaps to a back), a few plays are missing an event, and the 2025 files added a `frameType` column that marks the snap frame directly; `gridiron.bdb.prepare()` checks all three. **The 2026 files split each play at the throw.** The Big Data Bowl 2026 asked entrants to predict where players would go *while the ball was in the air*, so its `input_` files hold the frames up to the throw and its `output_` files hold the positions afterwards, with snake_case column names (`game_id`, `play_id`, `nfl_id`, `frame_id`, `play_direction`) instead of camelCase.⁴ `gridiron.bdb.load_tracking()` renames those, but stitching the two halves back together is your job.

Drawing and animating a play

Everything is now in place to look at a play. `gridiron.bdb.prepare()` does the steps above in one call for a single play: it picks the play's frames, standardises direction, marks each row as offense, defense or ball, and adds `time` in seconds from the snap. Its output is the input the drawing functions expect, whether the frames came from Kaggle or from a simulation. (The `one` table was built that way in the orientation figure's cell above, plus a few columns of our own: the `label` letters, and `d`, `w` and `frame_rel`.)

```
show_animation(one, times=[-1.5, 0.0, 1.5, T_ARR], to_go=10)
```

⁴Big Data Bowl 2025 tracking columns (camelCase: `gameId`, `playId`, `nflId`, `displayName`, `frameId`, `frameType`, `time`, `jerseyNumber`, `club`, `playDirection`, `x`, `y`, `s`, `a`, `dis`, `o`, `dir`, `event`) and 2026 columns (snake_case `input_*/output_*` files with `game_id`, `play_id`, `nfl_id`, `frame_id`, `play_direction`, ... `x`, `y`, `s`, `a`, `dir`, `o`, plus `num_frames_output`, `ball_land_x`, `ball_land_y`), as used in public solution code: <https://github.com/ZekeWeng/BigDataBowl2025> and <https://github.com/lneuendorf/nlf-big-data-bowl-2026>; competition overview at <https://operations.nfl.com/gameday/analytics/big-data-bowl>.

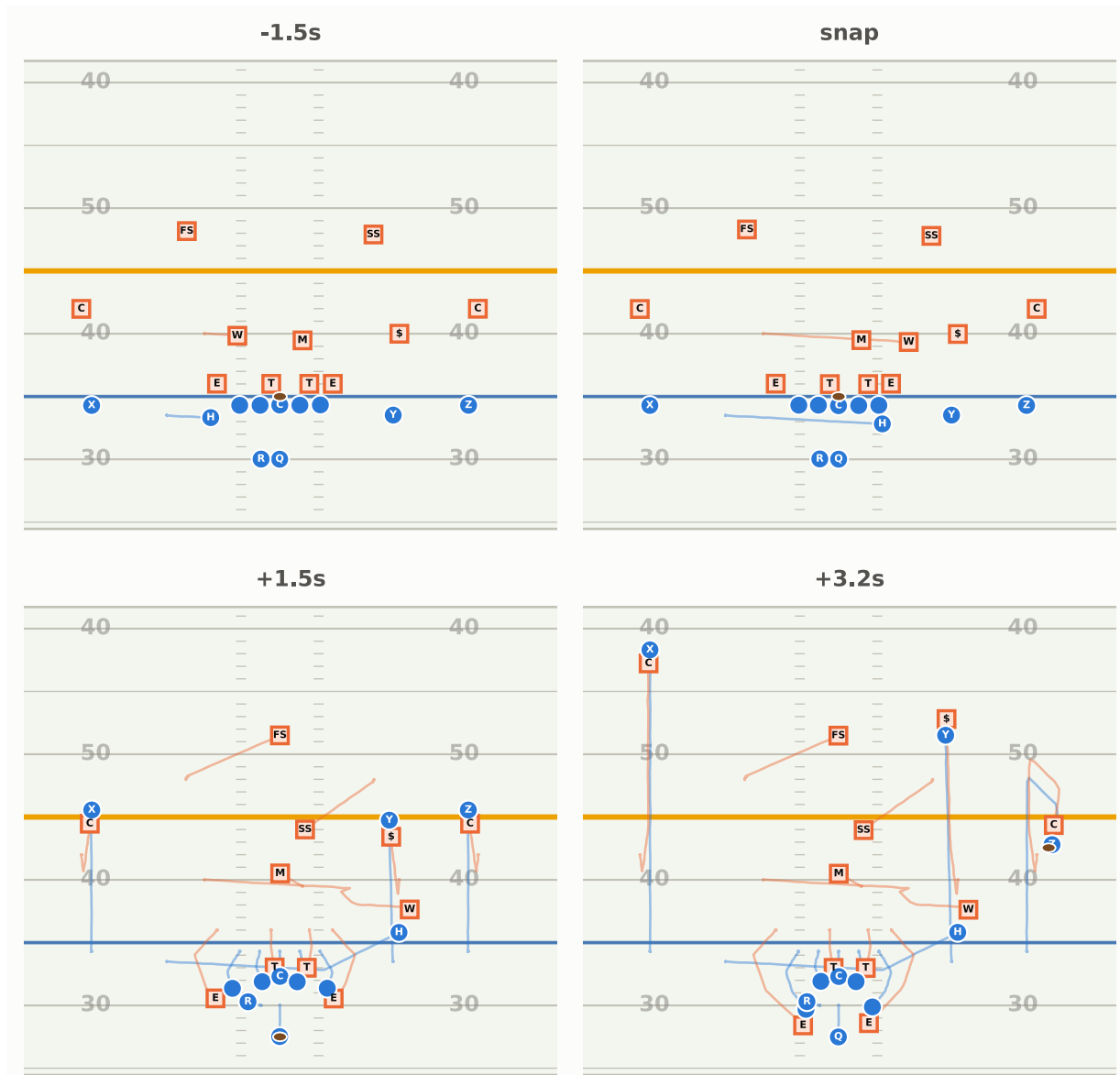


Figure 6: The chapter play, drawn from its tracking file (simulated, BDB-format; faint lines are each player's path so far). -1.5 s: two-high safeties, and H in motion from the left slot with the Will linebacker (W) travelling across with him, a sign of man coverage. Snap: H is still moving; nothing has rotated yet. +1.5 s: the rotation has happened after the snap: the left safety (FS) has run to the deep middle, the right safety (SS) has come down to about 9 yards in the middle as the robber, and the corners are running with their receivers. +3.2 s: Z has broken back toward the sideline and worked back to the ball, which arrives with the right corner closing a step and a half away.

On the website that is a video you can scrub; in print it is four frames. Either way, notice what the picture shows that a coach's diagram cannot: *timing*. The robber is in place by about one second; the quarterback releases the ball before Z has broken, and the ball is in the air for more than a second.

With a real Big Data Bowl 2025 download, the same picture of any play takes the same three calls.

```
# Download from https://www.kaggle.com/competitions/nfl-big-data-bowl-2025/data (accept the
# rules first) and unzip into data/bdb/. Files: games.csv, plays.csv, players.csv,
# player_play.csv and tracking_week_1.csv ... tracking_week_9.csv.
trk = bdb.load_tracking("data/bdb/tracking_week_1.csv")
plays = pd.read_csv("data/bdb/plays.csv")
players = pd.read_csv("data/bdb/players.csv")[["nflId", "position"]]
# pick a play by its description, e.g. the first charted Cover-1 dropback of week 1
```

```

in_week = plays["gameId"].isin(trk["gameId"].unique())
pick = plays[(plays["pff_passCoverage"] == "Cover-1") & in_week].iloc[0]
real = bdb.prepare(trk, plays, game_id=pick.gameId, play_id=pick.playId) # labels = jersey numbers
real = real.merge(players, on="nflId", how="left")
real["label"] = real["position"].fillna("") # or keep the numbers
print(pick.playDescription)
show_animation(real, times=[-1.5, 0.0, 1.5, 3.0])

```

Feature 1: separation

Separation is the distance from a receiver to the nearest defender. Next Gen Stats measures it at one moment, when the ball arrives (the catch or the incompletion), and averages it over a season.⁵ With frames you can measure it at *every* moment, which tells you much more than one number: when the receiver got open, and whether the defender closed the gap while the ball was in the air.

```

RECEIVERS = ["X", "H", "Y", "Z"]

def separation(frames, receivers=RECEIVERS):
    """Distance from each receiver to the nearest defender in every frame, and who that defender is."""
    dfn = frames.loc[frames["side"] == "def", ["frameId", "displayName", "x", "y"]]
    out = []
    for key in receivers:
        me = frames.loc[frames["displayName"] == key, ["frameId", "time", "x", "y"]]
        pair = me.merge(dfn, on="frameId", suffixes=("", "_def"))
        pair["sep"] = np.hypot(pair["x"] - pair["x_def"], pair["y"] - pair["y_def"])
        best = pair.loc[pair.groupby("frameId")["sep"].idxmin()]
        out.append(best.assign(receiver=key)[["frameId", "time", "receiver", "sep", "displayName"]])
    return pd.concat(out, ignore_index=True).rename(columns={"displayName": "nearest"})

sep = separation(one)
at = sep[np.isclose(sep["time"], T_THROW) | np.isclose(sep["time"], T_ARR)]
md_table(at.pivot(index="receiver", columns="time", values="sep")
         .rename(columns={T_THROW: "at the throw", T_ARR: "at the catch"}).reset_index())

```

receiver	at the throw	at the catch
H	2.04	1.99
X	1.13	1.09
Y	1.32	1.29
Z	0.82	1.57

```

fig, ax = plt.subplots(figsize=(6.5, 3.4))
show = sep[sep["time"].between(0, T_ARR + 0.6)]
for k, (key, g) in enumerate(show.groupby("receiver", sort=False)):
    ax.plot(g["time"], g["sep"], color=style.SERIES[k], lw=2.6 if key == "Z" else 1.4,
           label=f"{key} (the target)" if key == "Z" else key, zorder=3 if key == "Z" else 2)
for t_, lab in ((T_THROW, "throw"), (T_ARR, "catch")):
    ax.axvline(t_, color=style.INK_2, lw=1, ls=(0, (3, 2)))

```

⁵NFL Next Gen Stats defines average separation as the distance in yards between the receiver and the nearest defender at the time of the catch or incompletion (nflverse Next Gen Stats data dictionary: https://nflreadr.nflverse.com/articles/dictionary_nextgen_stats.html).

```

ax.text(t_, 6.6, f" {lab}", fontsize=8, color=style.INK_2, va="top")
ax.set_xlabel("Seconds after the snap")
ax.set_ylabel("Yards to the nearest defender")
ax.set_xlim(0, T_ARR + 0.7)
ax.set_ylim(0, 8.6)
ax.set_title("Separation, frame by frame")
ax.legend(loc="upper right", fontsize=8, ncol=4, frameon=True, facecolor=style.SURFACE,
        edgecolor="none", framealpha=1)
plt.show()

```

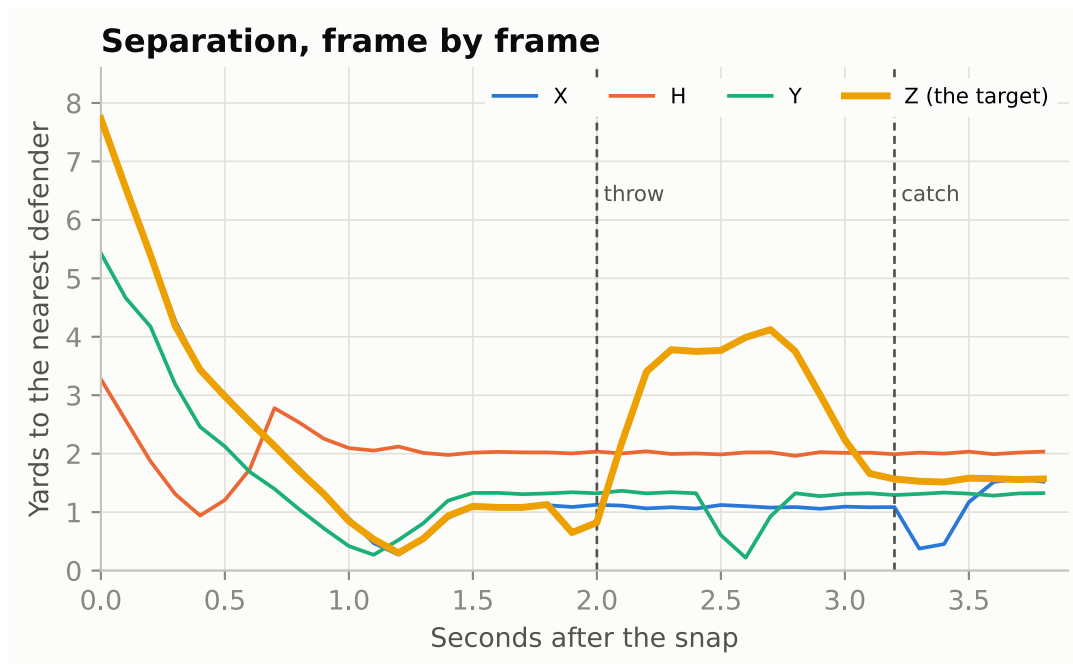


Figure 7: Separation over time for each receiver on the chapter play (simulated, BDB-format): the distance to the nearest defender, from the snap to just after the catch. Every line starts high because defenders begin 5 to 8 yards off, then drops as the cushions close. Z, the target, is blanketed (under a yard) when the ball is thrown, at the top of his stem; his break back toward the sideline opens four yards on the corner while the ball is in the air, and the corner has closed most of it again by the catch. H’s line stays near 2 yards: the Will linebacker who followed him across the formation trails him to the flat. The two vertical lines are the throw and the catch.

Two lessons hide in that chart.

Separation at the throw, at the catch and in between are three different numbers. The quarterback threw to a receiver who was not open yet: a timing throw, released before the break, trusting Z to win it. Z was most open in mid-flight, about four yards, and the corner had cut that to about a yard and a half by the catch. The NGS number you see on broadcasts is the last of the three. “Throwing receivers open” and “closing on the ball” are both real skills that a single snapshot hides, and measuring what happens while the ball is in the air was the whole point of the 2026 Big Data Bowl.

“Nearest defender” is not “covering defender.” On this play it is, because everyone is in man. In zone coverage the nearest defender may be responsible for a different receiver entirely, and a receiver running past a linebacker who is dropping to his own zone looks covered for a frame or two. The cleaner measure, “distance to the defender assigned to him”, needs an assignment, and assignments are exactly what tracking does not record (the Big Data Bowl 2025 files add PFF’s charted matchups in `player_play.csv` for that reason).

What season-long separation really measures

NGS publishes each qualifying receiver’s average separation. Here is the 2025 season, plotted against how far downfield each player’s targets were.

```

ngs = nfl().load_nextgen_stats(seasons=list(range(2016, 2026)), stat_type="receiving").to_pandas()
season = ngs[(ngs["week"] == 0) & (ngs["season_type"] == "REG")]
rec25 = season[season["season"] == 2025].copy()
slope, icpt = np.polyfit(rec25["avg_intended_air_yards"], rec25["avg_separation"], 1)
r_air = np.corrcoef(rec25["avg_intended_air_yards"], rec25["avg_separation"])[0, 1]
by_player = season.pivot_table(index="player_gsis_id", columns="season", values="avg_separation")
yoy = by_player[[2024, 2025]].dropna() # receivers who qualified in both seasons
r_yoy = yoy.corr().iloc[0, 1]
rec25["resid"] = rec25["avg_separation"] - (icpt + slope * rec25["avg_intended_air_yards"])

fig, ax = plt.subplots(figsize=(6.5, 3.9))
for k, pos in enumerate(["WR", "TE"]):
    g = rec25[rec25["player_position"] == pos]
    ax.scatter(g["avg_intended_air_yards"], g["avg_separation"], s=22, color=style.SERIES[k],
               edgecolor=style.SURFACE, lw=0.6, zorder=3,
               label={"WR": "Wide receivers", "TE": "Tight ends"}[pos])
xs = np.linspace(rec25["avg_intended_air_yards"].min(), rec25["avg_intended_air_yards"].max(), 10)
ax.plot(xs, icpt + slope * xs, color=style.INK_2, lw=1, ls=(0, (4, 2)), zorder=2)
low = rec25.nsmallest(2, "avg_separation")
high = rec25.nlargest(2, "avg_separation")
jsn = rec25[rec25["player_display_name"] == "Jaxon Smith-Njigba"]
offsets = [(14, -6), (14, 4), (6, 9), (14, -14), (12, 8)]
for (_, row), (ox, oy) in zip(pd.concat([low, high, jsn]).iterrows(), offsets):
    ax.annotate(f"{row['player_display_name']} ({row['team_abbr']})",
               (row["avg_intended_air_yards"], row["avg_separation"]), xytext=(ox, oy),
               textcoords="offset points", fontsize=7, color=style.INK,
               ha="left" if ox > 0 else "right", va="center",
               arrowprops=dict(arrowstyle="-", color=style.INK_2, lw=0.6))
ax.set_xlabel("Average intended air yards (how deep his targets were)")
ax.set_ylabel("Average separation (yd)")
ax.set_title("Separation is mostly a route-depth stat")
ax.set_ylim(None, rec25["avg_separation"].max() + 0.35)
ax.legend(loc="upper right", fontsize=8)
plt.show()

```

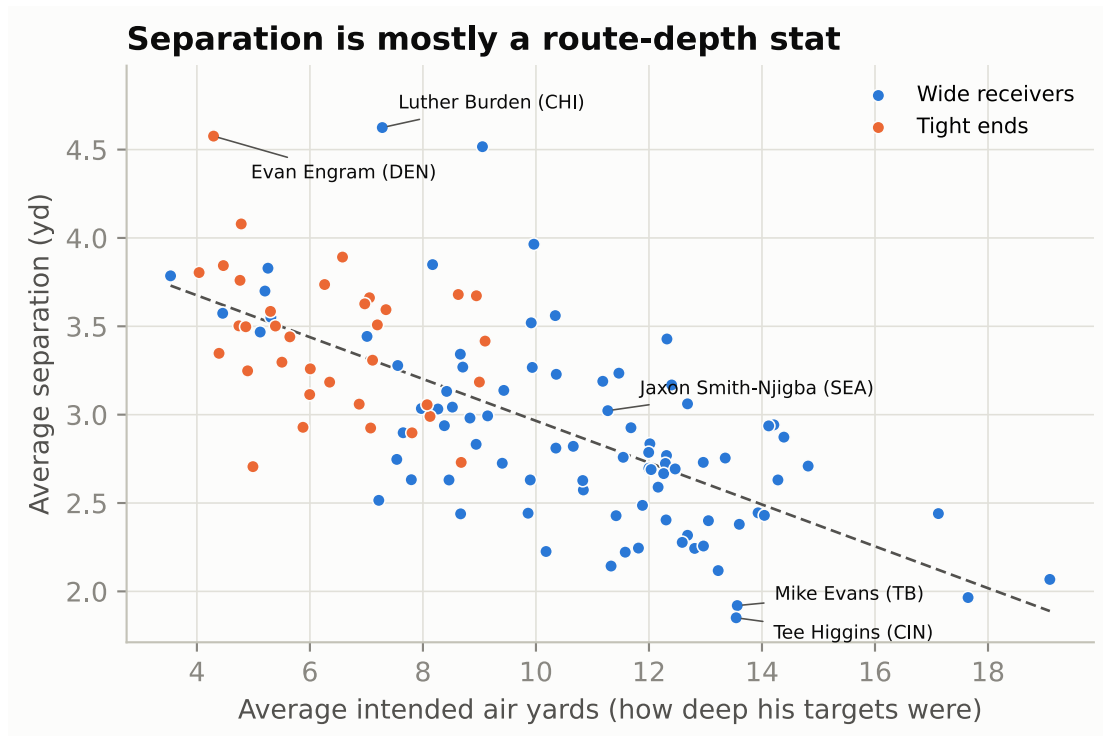


Figure 8: Average separation at the catch or incompletion against average depth of target (intended air yards), 2025 regular season, every receiver who qualified in Next Gen Stats’ season table. Real data: NFL Next Gen Stats via nflverse. Separation falls steadily the deeper the targets: deep routes are contested by design. Tight ends (orange) sit high and to the left because they are thrown short, often against linebackers. The labelled players are the season’s least and most separated receivers and the Offensive Player of the Year, Jaxon Smith-Njigba.

The correlation between depth of target and separation is -0.69 : every extra yard of target depth costs about 0.12 yards of separation. So before you call a receiver “always open”, ask how deep he runs. A fairer reading is the gap between a receiver’s separation and the dashed line, separation *for his depth*. That is the same “over expected” idea you met with RYOE and CPOE, done with one variable instead of a model. Separation is also fairly stable: for receivers who qualified in both 2024 and 2025, the year-to-year correlation is 0.73, partly because route depth is a role that carries over.

```
th = rec25[rec25["player_display_name"] == "Tee Higgins"].iloc[0]
least = rec25.nsmallest(4, "avg_separation")[["player_display_name", "team_abbr", "avg_separation",
                                              "avg_intended_air_yards", "targets", "catch_percentage",
                                              "yards"]]
md_table(least.set_axis(["player", "team", "separation (yd)", "target depth (yd)", "targets",
                        "catch %", "yards"], axis=1), 2)
```

player	team	separation (yd)	target	depth (yd)	targets	catch %	yards
Tee Higgins	CIN	1.85		13.54	98	60.2	846
Mike Evans	TB	1.92		13.56	62	48.39	368
Christian Watson	GB	1.97		17.65	55	63.64	611
Alec Pierce	IND	2.07		19.09	84	55.95	1003

The player: Tee Higgins, who had the lowest average separation of any qualifying receiver in 2025 (1.85 yards) and still caught 60% of his 98 targets for 846 yards, on targets averaging 13.5 yards downfield.⁶

What to notice: low separation is not the same as “not open.” Some receivers win *at the catch point*, with size, body position and hands, and their quarterbacks throw to them when they are covered. Watch a Bengals game and look at where the ball goes when Higgins is tightly covered on a fade or a back-shoulder throw. The number describes how he wins, not whether he does; that is what makes a tracking stat useful, and also what makes it easy to misread.

Feature 2: the pocket

Pass Protection drew the pocket as the shape enclosed by the five linemen and the quarterback (its *convex hull*, the smallest shape with no dents that contains all six), and showed it breathing over a dropback. Here is that idea as a number you can compute on every frame of every play, with a second number that answers the question the quarterback is actually asking: how close is the nearest defender?

```
def hull(points):
    """Convex hull of 2-D points (Andrew's monotone chain), as an array of corners, and its area."""
    pts = sorted(map(tuple, np.round(points, 4)))

    def half(seq):
        h = []
        for q in seq:
            while len(h) >= 2 and ((h[-1][0] - h[-2][0]) * (q[1] - h[-2][1])
                                   - (h[-1][1] - h[-2][1]) * (q[0] - h[-2][0])) <= 0:
                h.pop()
            h.append(q)
        return h

    corners = np.array(half(pts)[-1:] + half(pts[:-1])[-1:])
    x, y = corners[:, 0], corners[:, 1]
    area = 0.5 * abs(np.dot(x, np.roll(y, -1)) - np.dot(y, np.roll(x, -1))) # shoelace formula
    return corners, area

POCKET = ["LT", "LG", "C", "RG", "RT", "QB"]
rows = []
for fid, g in one[one["frame_rel"].between(0, 30)].groupby("frameId"):
    _, area = hull(g.loc[g["displayName"].isin(POCKET), ["x", "y"]].to_numpy())
    qb = g.loc[g["displayName"] == "QB", ["x", "y"]].to_numpy()[0]
    dfn = g.loc[g["side"] == "def", ["x", "y"]].to_numpy()
    rows.append((g["time"].iloc[0], area, np.hypot(*(dfn - qb).T).min()))
pocket = pd.DataFrame(rows, columns=["time", "area", "nearest"])
print(pocket.iloc[[0, 10, 20]].round(1).to_string(index=False))
```

time	area	nearest
0.0	13.7	6.0
1.0	20.0	5.5
2.0	16.7	3.1

⁶NFL Next Gen Stats season receiving table (2025 regular season, qualifying receivers), loaded with nflreadpy `load_nextgen_stats(stat_type="receiving")` and cached in `data/cache/`; data retrieved October 2026. Higgins's figures are this chapter's calculation from that table.

```

fig = plt.figure(figsize=(6.5, 3.4))
ax0 = fig.add_axes([0.0, 0.0, 0.44, 0.9])
ax1 = fig.add_axes([0.56, 0.56, 0.42, 0.36])
ax2 = fig.add_axes([0.56, 0.13, 0.42, 0.36], sharex=ax1)
fld = Field(ax=ax0, orient="vertical", los=LOS, window=(-10.5, 2.5),
            lateral=(BALL_Y - 7.5, BALL_Y + 7.5), numbers=False)
f_throw = int(one.loc[np.isclose(one["time"], T_THROW), "frameId"].iloc[0])
for t_, ls in ((0.0, (0, (1, 1.5))), (1.0, (0, (4, 2))), (T_THROW, "-")):
    g = one[np.isclose(one["time"], t_)]
    corners, _ = hull(g.loc[g["displayName"].isin(POCKET), ["x", "y"]].to_numpy())
    px, py = fld.xy(corners[:, 0], corners[:, 1])
    corners_px = np.c_[px, py]
    if t_ == T_THROW:
        # shade the pocket at the throw, under the players
        ax0.add_patch(Polygon(corners_px, fc=style.THIRD, alpha=0.18, ec="none", zorder=3))
        ax0.add_patch(Polygon(corners_px, fc="none", ec=style.THIRD, lw=1.6, ls=ls, zorder=12))
    g = one[(one["frameId"] == f_throw) & (one["side"] != "ball")
           & one["displayName"].isin(POCKET + ["RB", "WDE", "WDT", "SDT", "SDE"])]
draw_frame(fld, g)
ax0.set_title("Pocket hull: snap (dotted), +1 s (dashed), throw", fontsize=8, loc="left")
ax1.plot(pocket["time"], pocket["area"], color=style.THIRD, lw=2)
ax1.set_ylabel("Hull area (sq yd)", fontsize=8)
ax2.plot(pocket["time"], pocket["nearest"], color=style.SERIES[1], lw=2)
ax2.set_ylabel("Nearest defender\nto QB (yd)", fontsize=8)
ax2.set_xlabel("Seconds after the snap", fontsize=8)
for a_ in (ax1, ax2):
    a_.axvline(T_THROW, color=style.INK_2, lw=1, ls=(0, (3, 2)))
    a_.tick_params(labelsize=7.5)
ax1.text(T_THROW, ax1.get_ylim()[1], " throw", fontsize=7.5, color=style.INK_2, va="top")
plt.setp(ax1.get_xticklabels(), visible=False)
plt.show()

```

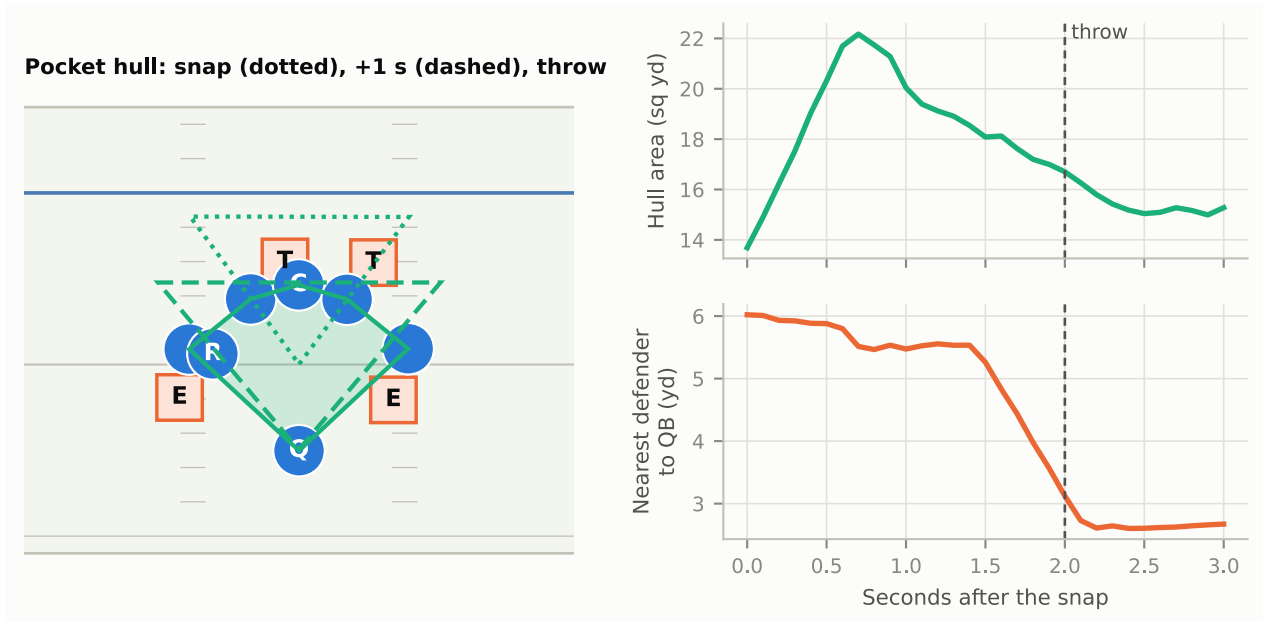


Figure 9: The pocket on the chapter play (simulated, BDB-format). Left: the convex hull of the five linemen and the quarterback at the snap (dotted), at +1.0 s (dashed) and at the throw (solid, shaded), with the linemen, the back and the four rushers at the throw. Right: the hull’s area and the distance from the quarterback to the nearest defender, frame by frame. The hull grows from about 14 to 22 square yards as the tackles kick back and the quarterback drops, then shrinks as the edge rushers push the tackles back toward him. The nearest defender is 5 to 6 yards away for the first one and a half seconds, about 3 yards at the throw, and closer still just after it. Neither number alone is pressure: a big hull can still have a free rusher inside it.

Each of these numbers is a simplification, and it is worth saying of what. The hull treats the pocket as the space *behind* the line, but quarterbacks feel pressure from the front: a defensive tackle driving the center back shrinks the space the quarterback can step into without changing the hull much. The nearest defender ignores who is blocking him: a rusher two yards away and locked up by a tackle is no threat, a blitzing three yards away and unblocked is. The tracking-based “win rates” ESPN publishes (pass rush win rate for one) solve that by asking a more specific question, whether a rusher has beaten his blocker within 2.5 seconds, and The Pass Rush shows how to compute get-off and time to pressure from the same frames.

Feature 3: motion at the snap

Motion and Shifts explained why offenses send a player across the formation, and why motion at the snap (a player still moving when the ball is snapped) has exploded: PFF counted shifts or motion on 63.9% of plays in 2025, up from 43.0% in 2018.⁷ Charting services decide by eye whether a play had motion, and their definitions differ: FTN’s `is_motion` flag in `nflverse` means motion “before or at” the snap, which is why its rates run lower than PFF’s. Tracking data lets you write your own definition and apply it the same way to every play.

The rule is short. **A player is in motion at the snap if he is moving faster than some threshold in the snap frame.** Linemen and the quarterback are excluded; they can’t legally be in motion, and their small movements are noise. A **shift** is different: several players reset, then everyone is still for a second before the snap. You can find it as displacement since the `line_set` event among players who are *stationary* at the snap.

```
SKILL = ["X", "H", "Y", "Z", "RB"]
THRESH = 1.0 # yards per second; wobble at rest is ~0.2-0.3
at_snap = one[(one["frameType"] == "SNAP") & one["displayName"].isin(SKILL)]
f_set = int(one.loc[one["event"] == "line_set", "frameId"].iloc[0])
start = one[(one["frameId"] == f_set) & one["displayName"].isin(SKILL)].set_index("displayName")
moved = np.hypot(at_snap.set_index("displayName")["x"] - start["x"],
                 at_snap.set_index("displayName")["y"] - start["y"])
```

⁷Daire Carragher, PFF, “NFL pre-snap motion usage reaches new highs across the NFL” (Feb. 23, 2026): shift/motion rate 43.0% in 2018 and 63.9% in 2025: <https://www.pff.com/news/nfl-pre-snap-motion-usage-reaches-new-highs-across-the-nfl>. FTN’s `is_motion` definition (“motion occurred on the play before or at the time of the snap”): https://nflreadr.nflverse.com/articles/dictionary_ftn_charting.html.

```
md_table(pd.DataFrame({"speed at snap": at_snap.set_index("displayName")["s"],
                      "yards moved since line set": moved,
                      "in motion at snap": at_snap.set_index("displayName")["s"] > THRESH}))
.reset_index().rename(columns={"displayName": "player"})
```

player	speed at snap	yards moved since line set	in motion at snap
RB	0.2	0.03	False
X	0.1	0.05	False
H	5.89	12.49	True
Y	0.07	0.03	False
Z	0.1	0.04	False

The defense's answer to motion is just as measurable, and more interesting. If a defender moves across the field *with* the motion man, that is a man-coverage tell, the idea behind the man/zone indicator: in zone, defenders bump over a few yards or stay put; in man, someone travels.

```
f_mot = int(one.loc[one["event"] == "man_in_motion", "frameId"].iloc[0])
win = one[one["frame_rel"].between(-30, 5)]
fig, axes = plt.subplots(2, 1, figsize=(6.5, 4.6), sharex=True)
ax = axes[0]
for key in SKILL:
    g = win[(win["displayName"] == key) & (win["frame_rel"] <= 0)]
    is_h = key == "H"
    ax.plot(g["frame_rel"], g["s"], color=style.OFFENSE if is_h else style.BASELINE,
            lw=2.2 if is_h else 1.0, zorder=3 if is_h else 2)
ax.text(-19.5, 6.6, "H, in motion", color=style.OFFENSE, fontsize=8, fontweight="bold")
ax.text(-19.0, 2.5, "X, Y, Z and the back: sensor wobble only", fontsize=7.5, color=style.MUTED)
ax.axhline(THRESH, color=style.INK_2, lw=0.9, ls=(0, (3, 2)))
ax.text(-6.0, THRESH + 0.25, "threshold, 1 yd/s", fontsize=7.5, color=style.INK_2)
ax.set_ylim(0, 8)
ax.set_ylabel("Speed (yd/s)", fontsize=8.5)
ax = axes[1]
base = win[win["frameId"] == f_mot].set_index("displayName")["y"]
for key in ["H", "WILL", "MIKE", "NB", "SS", "FS"]:
    g = win[win["displayName"] == key]
    across = (base[key] - g["y"]).to_numpy() # + = toward the offense's right
    is_h, is_w = key == "H", key == "WILL"
    col = style.OFFENSE if is_h else (style.DEFENSE if is_w else style.BASELINE)
    ax.plot(g["frame_rel"], across, color=col, lw=2.2 if (is_h or is_w) else 1.0,
            ls="-" if not is_h else (0, (4, 2)))
    if is_h or is_w:
        ax.text(5.6, across[-1], "H" if is_h else "Will", color=col, fontsize=8, fontweight="bold",
                va="center")
ax.text(-29.5, 1.0, "other defenders", fontsize=7.5, color=style.MUTED)
ax.set_ylabel("Yards moved across\nsince motion began", fontsize=8.5)
ax.set_xlabel("Frames from the snap (10 frames = 1 second)", fontsize=8.5)
for a_ in axes:
    a_.axvline(0, color=style.INK_2, lw=1)
    a_.set_xlim(-30, 8)
axes[0].text(0.4, 7.6, "snap", fontsize=7.5, color=style.INK_2, va="top")
plt.show()
```

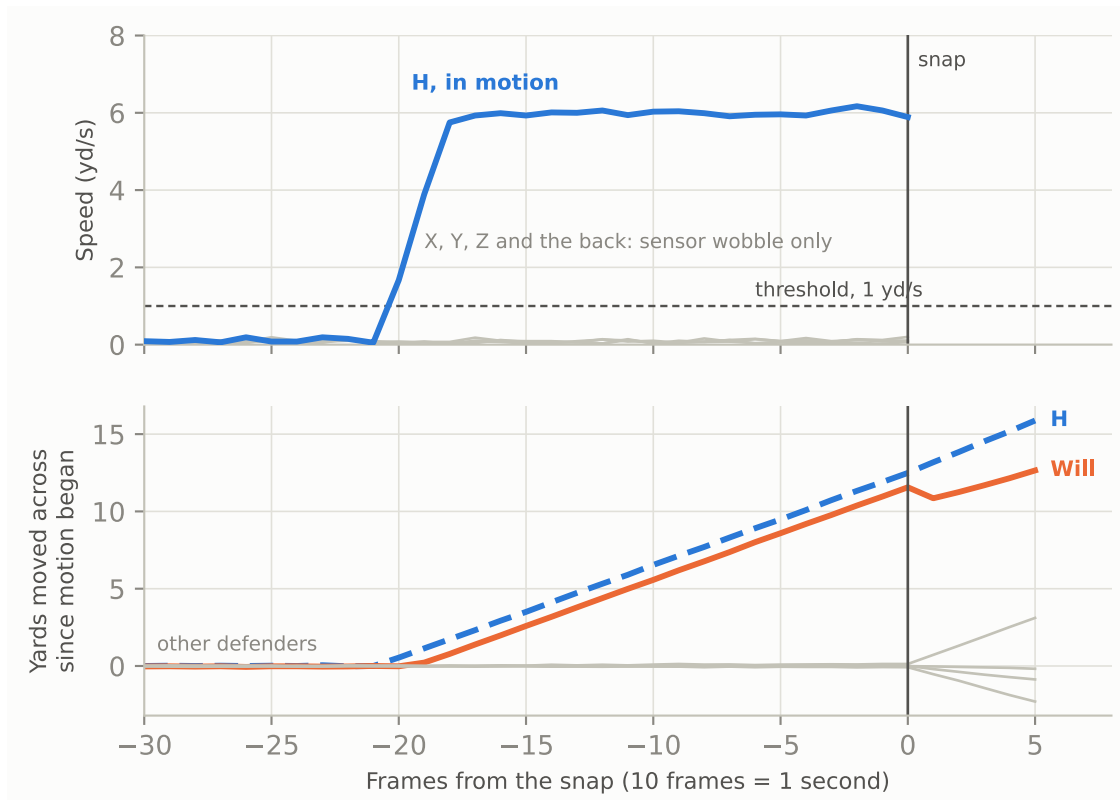


Figure 10: Motion, measured (the chapter play; simulated, BDB-format). Top: speed of each skill player over the last three seconds before the snap and the first half-second after. Everyone sits in the noise band near zero except H, who starts his motion about two seconds before the snap and is running at about 6 yards a second when the ball is snapped (frame 0): in motion at the snap. Bottom: how far each defender has moved across the field since H started. The Will linebacker moves almost exactly as far as H does (he travels with him, man coverage); the other defenders barely move.

That bottom panel is the seed of the first project in this chapter's last section: a man/zone classifier is, at heart, a careful version of "did anyone follow?". With the real files you can check your rule against the charters: the Big Data Bowl 2025 `player_play.csv` has `inMotionAtBallSnap`, `motionSinceLineset` and `shiftSinceLineset` for every player on every play.

```
pp = pd.read_csv("data/bdb/player_play.csv")
snap = trk[trk["frameType"] == "SNAP"].merge(players, on="nflId")
snap = snap[snap["position"].isin(["WR", "TE", "RB", "FB"])]
mine = snap.assign(my_motion=snap["s"] > 1.0)[["gameId", "playId", "nflId", "my_motion"]]
keys = ["gameId", "playId", "nflId"]
check = mine.merge(pp[keys + ["inMotionAtBallSnap"]], on=keys)
print(pd.crosstab(check["my_motion"], check["inMotionAtBallSnap"].fillna(False)))
# then tune the threshold (try 0.6 to 2.0 yd/s) and look at the plays where you disagree
```

Feature 4: safety rotation, step by step

This is the figure Disguise, Rotation, and Split-Field Coverage showed as a finished picture. Its job is to show *when* a defense's real coverage appears. Here it is built in four steps.

Step 1: find the safeties, and follow them on one play

The first decision is the least obvious: **who counts as a safety?** Not whoever the roster calls one. Teams play three-safety nickel packages, corners rotate to the deep middle on some calls, and a safety walked down into the box before the snap is not playing safety on that play. A robust definition uses the data itself: **the two deepest defenders one second before the snap**. Then measure, for each frame, how deep each one is (yards past the line of scrimmage) and how far he is sideways from the middle of the field.

```
def safety_tracks(frames, los, ball_y, lo=-30, hi=20):
    """The two deepest defenders 1 s before the snap: depth and distance from the ball, by frame."""
    f = frames.copy()
    f["frame_rel"] = np.round(f["time"] * FPS).astype(int)
    pre = f[(f["side"] == "def") & (f["frame_rel"] == -10)]
    ids = pre.nlargest(2, "x")["displayName"]
    s = f[f["displayName"].isin(ids) & f["frame_rel"].between(lo, hi)].copy()
    s["depth"] = s["x"] - los
    s["from_middle"] = (s["y"] - ball_y).abs()
    return s

saf1 = safety_tracks(one, LOS, BALL_Y)
print("safeties found:", sorted(saf1["displayName"].unique()))
```

```
safeties found: ['FS', 'SS']
```

```
fig, axes = plt.subplots(2, 1, figsize=(6.5, 4.0), sharex=True)
PANELS = ((0, "depth", "Depth (yd)", (1, "from_middle", "From the middle (yd)"))
for i_, col, ylab in PANELS:
    ax = axes[i_]
    ax.axvspan(-30, 0, color=style.GRID, alpha=0.35, lw=0)
    for k, (key, g) in enumerate(saf1.groupby("displayName")):
        ax.plot(g["frame_rel"], g[col], color=style.SERIES[k + 1], lw=2.2)
        ax.text(20.6, g[col].iloc[-1], key, color=style.SERIES[k + 1], fontsize=8, fontweight="bold",
                va="center")
    ax.axvline(0, color=style.INK_2, lw=1)
    ax.set_ylabel(ylab, fontsize=8.5)
    ax.set_xlim(-30, 25)
top = axes[0].get_ylim()[1] - 0.3
axes[0].text(-29, top, "before the snap", fontsize=8, color=style.INK_2, va="top")
axes[0].text(0.6, top, "after the snap", fontsize=8, color=style.INK_2, va="top")
axes[1].set_xlabel("Frames from the snap (10 frames = 1 second)", fontsize=8.5)
fig.tight_layout()
plt.show()
```

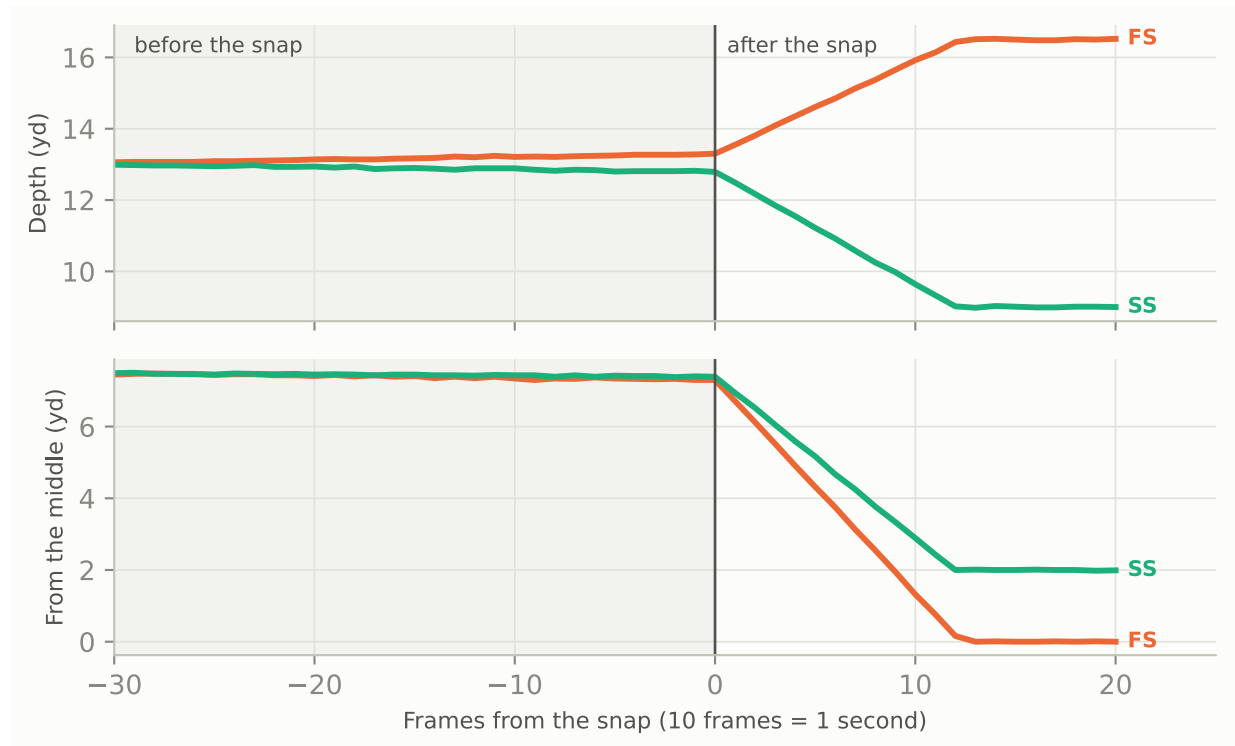


Figure 11: Step 1: the two safeties on the chapter play alone (simulated, BDB-format), from 3 seconds before the snap to 2 seconds after. Top: depth past the line of scrimmage. Bottom: sideways distance from the middle of the field (here the ball sits in the middle). Before the snap both sit about 13 yards deep and 7 to 8 yards off the middle: two-high. At the snap they split: the left safety (FS) goes deeper and runs to the middle (his sideways distance falls to zero), the right safety (SS) comes down to about 9 yards and pinches toward the middle: one-high, with a robber.

Step 2: many plays, one chart

One play is an anecdote. The chart in 06-06 stacks sixty: thirty on which a two-high shell stayed two-high, and thirty on which it rotated to one-high. The generator below is the one that chapter used, reproduced here with the same random seed so the figures match: it varies where the safeties start, which side the rotation goes to, how fast they move and, crucially, *when* they start moving (τ , in seconds; negative means before the snap).

```
def nickel_vs_2x2(shell="two_high", safety_w=7.5):
    """11 personnel 2x2 against a nickel (4-2-5); two-high safeties 13 deep, outside the hashes."""
    off = offense("gun_2x2")
    dfn = defense("nickel", shell, off)
    out = []
    for q in dfn:
        if q.key == "WILL":
            q = q.moved(d=5.0, w=-6.0)
        elif q.pos == "CB":
            q = q.moved(d=7.0, w=q.w + (0.8 if q.w > 0 else -0.8), label="CB")
        elif q.key in ("FS", "SS") and shell == "two_high":
            q = q.moved(d=13.0, w=safety_w if q.w > 0 else -safety_w)
        out.append(q)
    return off, out

def _seg(p, key, start, end, tau, speed):
    """Move `key` from start to end at `speed`, starting at time tau (negative = before the snap)."""
    start, end = np.array(start, float), np.array(end, float)
    L = float(np.linalg.norm(end - start))
```

```

if tau < 0:
    # part of the trip happens before the snap
    pre = min(L, speed * -tau)
    mid = start + (end - start) * (pre / L if L else 0)
    p.motion(key, to=tuple(mid), speed=max(pre / -tau, 0.05))
    if L - pre > 0.05:
        p.drop(key, tuple(end), speed=speed)
else:
    p.drop(key, tuple(end), speed=speed, delay=tau)

def sim_dropback(kind, rng, tau=None, play_id=0):
    """One simulated dropback as BDB-style tracking rows (10 frames a second).

    kind: 'stay2' (two-high shown and played), 'rot1' (two-high shown, rotated to one-high),
          'stay1' (one-high shown and played), 'spin2' (one-high shown, bailed to two-high).
    """
    off, dfn = nickel_vs_2x2()
    side = rng.choice([-1, 1])
    if kind in ("stay2", "rot1"):
        d1, d2 = rng.uniform(11.5, 14.0, 2)
        w1, w2 = rng.uniform(5.5, 8.5, 2)
        fs0, ss0 = (d1, -side * w1), (d2, side * w2)
    else:
        fs0 = (rng.uniform(13.0, 15.5), rng.uniform(-1.5, 1.5))
        ss0 = (rng.uniform(7.0, 10.0), side * rng.uniform(4.0, 7.0))
    dfn = [q.moved(d=fs0[0], w=fs0[1]) if q.key == "FS" else q for q in dfn]
    dfn = [q.moved(d=ss0[0], w=ss0[1]) if q.key == "SS" else q for q in dfn]
    p = Play(off, dfn, los=35, to_go=10)
    p.motion("MIKE", to=(4.5, 1.5), speed=0.1) # a slow shuffle so frames start 3 s early
    p.route("X", r.go(22)); p.route("Z", r.go(22)); p.route("H", r.seam(20)); p.route("Y", r.seam(20))
    p.dropback(2.5)
    tau = rng.uniform(-0.9, 0.3) if tau is None else tau
    v = rng.uniform(5.0, 6.5)
    t0 = max(tau, 0)
    if kind == "stay2":
        _seg(p, "FS", fs0, (fs0[0] + rng.uniform(2, 4), fs0[1] - side * rng.uniform(0.5, 2.5)),
            t0, v * .6)
        _seg(p, "SS", ss0, (ss0[0] + rng.uniform(2, 4), ss0[1] + side * rng.uniform(0.5, 2.5)),
            t0, v * .6)
    elif kind == "rot1":
        _seg(p, "FS", fs0, (rng.uniform(15.5, 18.0), rng.uniform(-1.0, 1.0)), tau, v)
        _seg(p, "SS", ss0, (rng.uniform(8.0, 10.5), side * rng.uniform(3.0, 6.0)), tau, v * 0.8)
    elif kind == "stay1":
        _seg(p, "FS", fs0, (fs0[0] + rng.uniform(2, 4), fs0[1]), t0, v * 0.6)
        _seg(p, "SS", ss0, (ss0[0] + rng.uniform(-1, 1), ss0[1] + side * rng.uniform(1, 4)),
            t0, v * 0.6)
    else:
        _seg(p, "FS", fs0, (rng.uniform(15.5, 18.0), -side * rng.uniform(9.0, 12.0)), tau, v)
        _seg(p, "SS", ss0, (rng.uniform(15.5, 18.0), side * rng.uniform(9.0, 12.0)), tau, v)
    trk_ = p.to_tracking(fps=FPS, play_id=play_id)
    trk_["kind"], trk_["los"], trk_["ball_y"] = kind, p.los, p.ball_y
    return trk_

```

The loop applies `safety_tracks()` from step 1 to each play, then labels each safety's **role**: on rotated plays, the one who finishes deepest "spun" to the middle and the other "came down"; on the others both "stayed".

```

rng = np.random.default_rng(2026)
taus = [0.0] * 30 + list(rng.uniform(0.0, 0.25, 22)) + list(rng.uniform(-1.0, -0.5, 8))
tracks = []
for i, kind in enumerate(["stay2"] * 30 + ["rot1"] * 30):
    t = sim_dropback(kind, rng, tau=taus[i], play_id=i + 1)
    s = safety_tracks(t, los=35, ball_y=t["ball_y"].iloc[0])
    tracks.append(s.assign(kind=kind))
saf = pd.concat(tracks, ignore_index=True)
end = saf[saf["frame_rel"] == 20].set_index(["playId", "displayName"])["depth"]
deepest = end.groupby(level=0).idxmax().map(lambda t: t[1])
saf["role"] = np.where(saf["kind"] == "stay2", "stayed",
                      np.where(saf["displayName"] == saf["playId"].map(deepest), "spun", "dropped"))
COL = {"stayed": style.SERIES[0], "spun": style.SERIES[1], "dropped": style.SERIES[2]}
LAB = {"stayed": "stayed two-high", "spun": "ran to the middle", "dropped": "came down"}

fig, axes = plt.subplots(2, 1, figsize=(6.5, 5.6), sharex=True)
for ax, col, ylab in ((axes[0], "depth", "Depth past the line of scrimmage (yd)"),
                     (axes[1], "from_middle", "Sideways from the middle (yd)")):
    ax.axvspan(-30, 0, color=style.GRID, alpha=0.35, lw=0)
    for _, g in saf.groupby(["playId", "displayName"]):
        ax.plot(g["frame_rel"], g[col], color=COL[g["role"].iloc[0]], lw=0.6, alpha=0.35)
    for role, g in saf.groupby("role"):
        med = g.groupby("frame_rel")[col].median()
        ax.plot(med.index, med.values, color=COL[role], lw=2.4)
        v = med.iloc[-1] + ({ "spun": 0.7, "stayed": -0.7 }.get(role, 0.0) if col == "depth" else 0.0)
        ax.text(20.6, v, LAB[role], color=COL[role], fontsize=8, va="center", fontweight="bold")
    ax.axvline(0, color=style.INK_2, lw=1.0)
    ax.set_ylabel(ylab, fontsize=8.5)
    ax.set_xlim(-30, 30)
top = axes[0].get_ylim()[1] - 0.3
axes[0].text(-29, top, "before the snap", fontsize=8, color=style.INK_2, va="top")
axes[0].text(0.6, top, "after the snap", fontsize=8, color=style.INK_2, va="top")
axes[1].set_xlabel("Frames from the snap (10 frames = 1 second)", fontsize=8.5)
axes[0].set_title("The two safeties around the snap (simulated, BDB-format tracking)", fontsize=10)
fig.tight_layout()
plt.show()

```

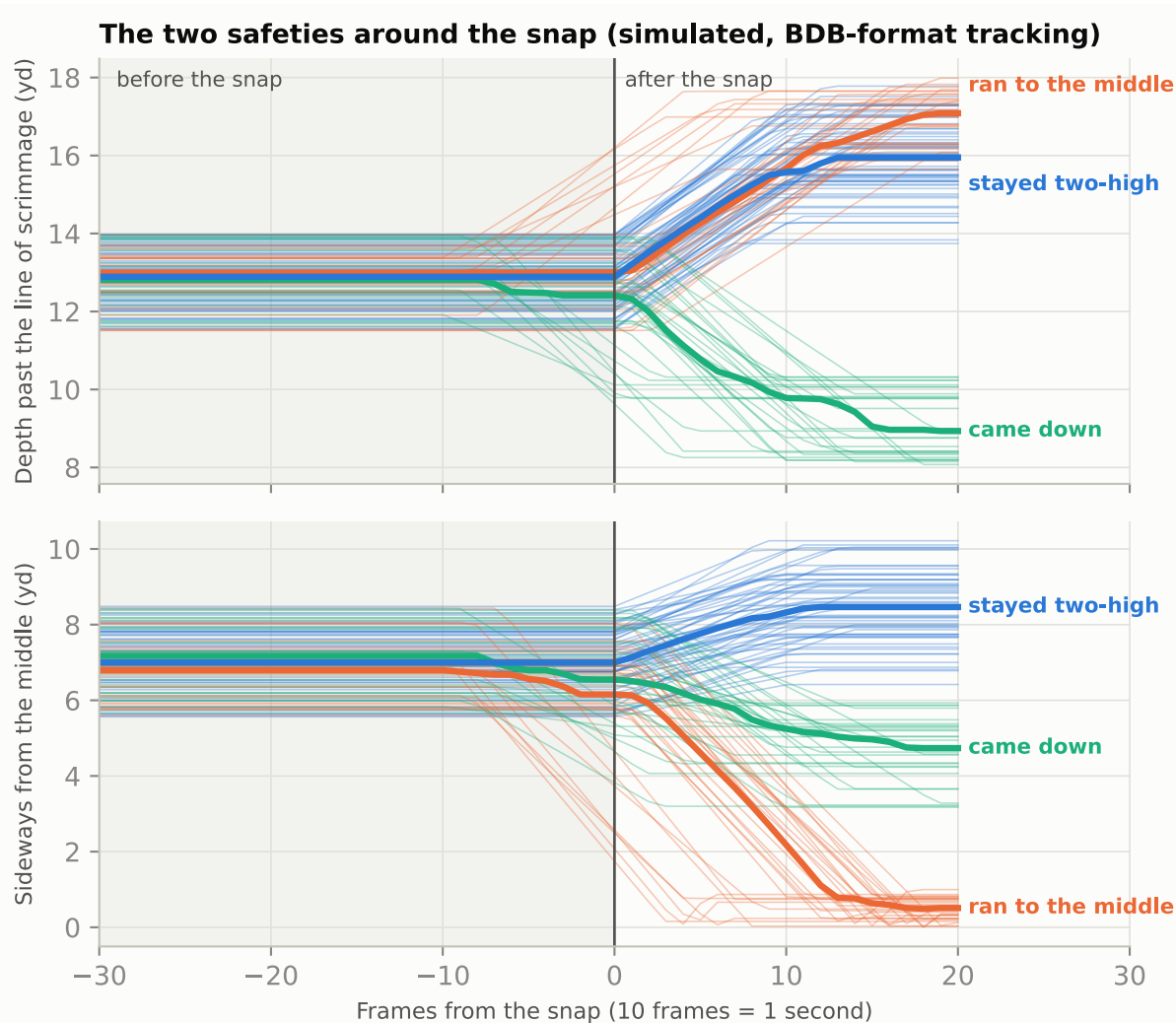


Figure 12: Step 2, the book’s safety-rotation figure (simulated, BDB-format tracking; not real plays): the two safeties on 60 plays that all showed two-high, from 30 frames (3 seconds) before the snap to 20 frames after. Thin lines are single plays; thick lines are medians. Top: depth. Bottom: sideways distance from the middle of the field. On 30 plays the shell stayed two-high (blue): both safeties drift back and a little wider. On the other 30 it rotated: one safety ran to the middle (orange: his sideways distance falls to about zero) while the other came down to robber depth (green: depth falls to about 9 or 10 yards, and he pinches in toward the hole). 22 of the 30 rotations were set to start at the snap (late rotation), so the thick lines barely move before frame 0 and split sharply after it; the other 8 start 5 to 10 frames early, and their thin lines peel away before the snap: rotations the quarterback could have seen.

Read it as the shape to look for, not as evidence: a simulation gives back only the timings it was given. A rotation is a **divergence**. Before it starts, rotated and honest plays are indistinguishable; within about ten frames they are in completely different places. Everything a defense coordinator means by “rotate late” is about where that divergence begins.

Step 3: when did it rotate?

The chart shows timing by eye. To compare defenses you need it as a number for every play: the **rotation onset**, the frame at which the safety who finishes in the middle starts moving there. The rule below first decides whether the play rotated at all (one safety ends within 2.5 yards of the middle and the other ends shallower than 11 yards), then finds the first frame at which the middle safety is a yard closer to the middle than he was during the cadence.

```
def rotation_onset(s):
    """Frames from the snap at which the safety who ends in the middle starts moving there.
    None if the play never rotated to one-high (s: one play's output of safety_tracks)."""
    last = s[s["frame_rel"] == s["frame_rel"].max()].set_index("displayName")
    post = last["from_middle"].idxmin()
```

```

other = last.drop(index=post)
if last.loc[post, "from_middle"] > 2.5 or other["depth"].iloc[0] > 11.0:
    return None
g = s[s["displayName"] == post]
base = g.loc[g["frame_rel"].between(-30, -20), "from_middle"].median()
return int(g.loc[g["from_middle"] < base - 1.0, "frame_rel"].min())

onsets = (saf.groupby("playId").apply(rotation_onset, include_groups=False).dropna().astype(int)
        .rename("onset").reset_index())
onsets["kind"] = onsets["playId"].map(saf.drop_duplicates("playId").set_index("playId")["kind"])
n_late, n_early = (onsets["onset"] >= 0).sum(), (onsets["onset"] < 0).sum()
print(f"{len(onsets)} of 60 plays flagged as rotations; "
      f"{n_late} started at or after the snap, {n_early} before it")

```

30 of 60 plays flagged as rotations; 22 started at or after the snap, 8 before it

```

fig, ax = plt.subplots(figsize=(6.5, 1.9))
jit = np.random.default_rng(1).uniform(-0.3, 0.3, len(onsets))
ax.axvspan(-5, 0, color=style.GRID, alpha=0.6, lw=0)
late = onsets["onset"] >= 0
ax.scatter(onsets["onset"] + jit * 0.4, jit, s=30, edgecolor=style.SURFACE, lw=0.6, zorder=3,
          color=np.where(late, style.SERIES[1], style.SERIES[0]))
ax.axvline(0, color=style.INK_2, lw=1)
ax.text(1.0, 0.75, f"late: {late.sum()} plays", color=style.SERIES[1], fontsize=8, fontweight="bold")
ax.text(-15.5, 0.75, f"early: {(~late).sum()} plays", color=style.SERIES[0], fontsize=8,
       fontweight="bold")
ax.text(-2.5, -0.85, "last 0.5 s", color=style.INK_2, fontsize=7.5, ha="center")
ax.set_ylim(-1.1, 1.1)
ax.set_yticks([])
ax.set_xlim(-16, 8)
ax.set_xlabel("Rotation onset, frames from the snap", fontsize=8.5)
ax.grid(axis="y", visible=False)
plt.show()

```

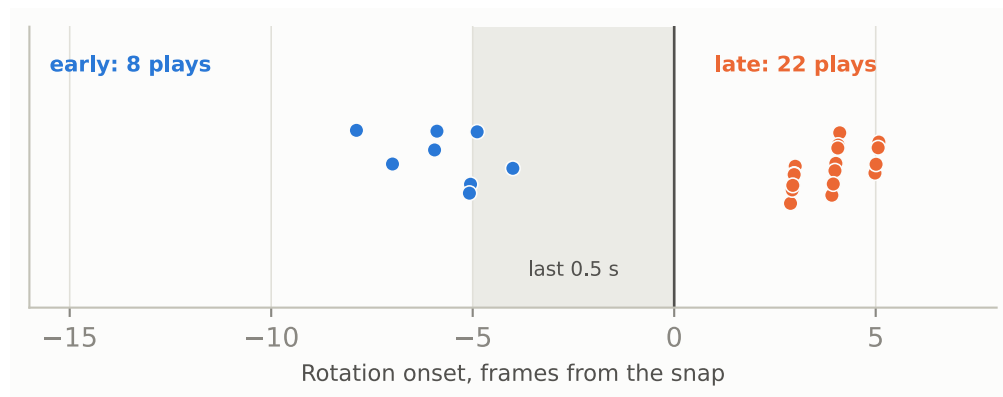


Figure 13: Step 3: rotation onset for every play the rule flagged as a rotation (simulated, BDB-format). Each dot is one play, placed at the frame at which its middle safety had moved a yard toward the middle. The shaded band marks the last half-second before the snap: a quarterback who sees the rotation start there can still change the protection or his first read, but rarely the play. The rule found all 30 rotations and none of the 30 honest plays. The 22 late rotations register 3 to 5 frames after the snap, and the 8 early ones 4 to 8 frames before it; in both groups the rule fires two or three frames after the safety really started, the time it takes him to cover the yard.

With rotation onset as a column, the measurement 06-06 sketched becomes a few lines of pandas: the share of a defense's two-high looks that rotate, and the median onset of those rotations. The first number is *how often it lies*; the second is *how late*. A defense with both high is the hardest to read before the snap, which is the point of late rotation.

Step 4: the same thing on real files

```
# BDB 2025: 2022 season, Weeks 1-9, every frame from the huddle break to the end of the play.
plays = pd.read_csv("data/bdb/plays.csv")
dropbacks = plays[plays["pff_passCoverage"].notna()]
frames, onset_rows = [], []
for week in range(1, 10): # one week at a time keeps memory sane
    trk = bdb.load_tracking(f"data/bdb/tracking_week_{week}.csv")
    trk = trk.merge(dropbacks[["gameId", "playId"]], on=["gameId", "playId"])
    for (g, pl), _ in trk.groupby(["gameId", "playId"]):
        one = bdb.prepare(trk, plays, game_id=g, play_id=pl) # standardised, time 0 = snap
        snap = one[(one["frameType"] == "SNAP") & (one["side"] == "ball")]
        if snap.empty or one["time"].min() > -3.0: # need three seconds of pre-snap frames
            continue
        s = safety_tracks(one, float(snap["x"].iloc[0]), float(snap["y"].iloc[0]))
        frames.append(s.assign(gameId=g, playId=pl))
        onset_rows.append((g, pl, rotation_onset(s)))
saf = pd.concat(frames).merge(plays[["gameId", "playId", "defensiveTeam", "pff_passCoverage"]],
                             on=["gameId", "playId"])
onsets = pd.DataFrame(onset_rows, columns=["gameId", "playId", "onset"])
# Plot `saf` exactly as in step 2 (role from the coverage label and the deepest safety at +20),
# and group `onsets` by defensiveTeam for each defense's rotation rate and median onset.
```

Expect the real version to be messier, in useful ways. Real safeties bounce, take false steps and react to motion, so the thin lines will be noisy. Plays start at different distances from the snap, so some will have fewer than thirty pre-snap frames. And the role labels will need the charted coverage, because a real defense can rotate *and* play two-high (a “buzz” or “cloud” rotation changes who has which zone without changing the shell).

Safety depth at the snap, team by team

The shell a defense shows at the snap can be boiled down to one number per play: **the depth of the second-deepest defender**. With two high safeties it is about 12 to 14 yards; with one, the second-deepest defender is a safety in the box or a corner, 7 to 10 yards deep. So a team's distribution of that number is a portrait of its shell habits. The chart below draws that portrait for all 32 defenses. The depths are simulated, but each team's mix of shells is real: its share of two-high coverages (Cover 2, 2-Man, Cover 4, Cover 6) among charted dropbacks in Weeks 1 to 9 of 2022, the slice of the 2022 season covered by the 2025 Big Data Bowl files, from nflverse's participation data.⁸

```
part = nfl().load_participation(seasons=[2022]).to_pandas()
pbp22 = (nfl().load_pbp([2022])
         .select(["game_id", "play_id", "defteam", "season_type", "week"]).to_pandas())
cov = part.merge(pbp22, left_on=["nflverse_game_id", "play_id"], right_on=["game_id", "play_id"])
cov = cov[(cov["season_type"] == "REG") & (cov["week"] <= 9)
          & cov["defense_coverage_type"].notna() & (cov["defense_coverage_type"] != "PREVENT")]
cov["two_high"] = cov["defense_coverage_type"].isin(["COVER_2", "2_MAN", "COVER_4", "COVER_6"])
share = cov.groupby("defteam")["two_high"].mean().sort_values()

rng = np.random.default_rng(22)
pts = []
for team, p2 in share.items():
```

⁸nflverse participation data (load_participation, 2022 season; coverage labels supplied by NGS for 2016-2022), joined to nflverse play-by-play for the defensive team and week; regular season, Weeks 1-9, dropbacks with a coverage label other than PREVENT. Two-high = COVER_2, 2_MAN, COVER_4 and COVER_6. This chapter's calculation; data retrieved October 2026.

```

for i in range(30):
    kind = "stay2" if rng.random() < p2 else "stay1"
    t = sim_dropback(kind, rng, tau=0.0)
    snap = t[(np.isclose(t["time"], 0.0)) & (t["side"] == "def")]
    second = np.sort((snap["x"] - 35).to_numpy())[-2]
    pts.append((team, second))
depth = pd.DataFrame(pts, columns=["team", "second_deepest"])

fig, ax = plt.subplots(figsize=(6.5, 7.2))
ypos = {t: i for i, t in enumerate(share.index)}
yj = np.random.default_rng(3).uniform(-0.25, 0.25, len(depth))
two = depth["second_deepest"] > 11
ax.scatter(depth["second_deepest"], depth["team"].map(ypos) + yj, s=9,
           color=np.where(two, style.SERIES[0], style.SERIES[1]), alpha=0.75, lw=0, zorder=3)
ax.axvline(11, color=style.INK_2, lw=0.8, ls=(0, (3, 2)))
for t, i in ypos.items():
    ax.text(14.9, i, f"{share[t]:.0%}", fontsize=7, color=style.INK_2, va="center")
ax.text(14.9, len(ypos) + 0.2, "two-high\nshare (real)", fontsize=7, color=style.INK_2, va="bottom")
for x_, txt, col in ((8.3, "one-high", style.SERIES[1]), (13.0, "two-high", style.SERIES[0])):
    ax.text(x_, len(ypos) + 0.2, txt, fontsize=8, color=col, ha="center", fontweight="bold")
ax.set_yticks(list(ypos.values()), list(ypos.keys()), fontsize=7.5)
ax.set_ylim(-0.8, len(ypos) + 1.6)
ax.set_xlim(6, 16.4)
ax.set_xticks([6, 8, 10, 12, 14])
ax.set_xlabel("Depth of the second-deepest defender at the snap (yd)", fontsize=8.5)
ax.grid(axis="y", visible=False)
plt.show()

```

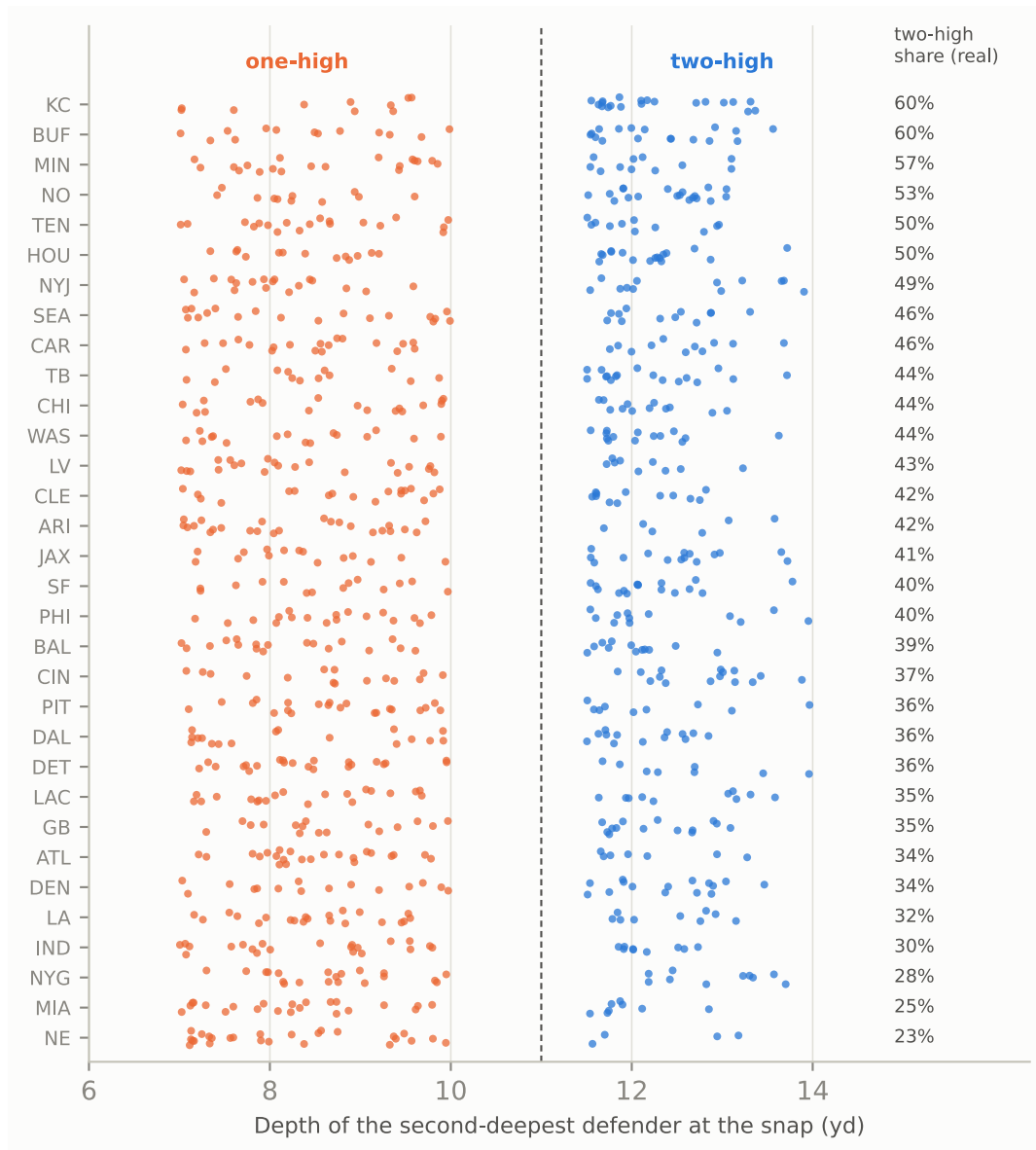


Figure 14: Depth of the second-deepest defender at the snap, 30 simulated dropbacks per defense (simulated, BDB-format depths), with each defense's share of two-high shells set to its real share of two-high coverages on charted dropbacks in Weeks 1-9 of 2022 (nflverse participation data). Dots above about 11 yards (blue) are two-high looks; dots below (orange) are one-high. The distribution splits into two clusters for every team; what separates teams is how the dots divide between the clusters, from New England and Miami, mostly one-high, to Kansas City and Buffalo, mostly two-high. With the real files the clusters would blur (disguise, field position, the hash), which is exactly what makes the real version worth computing.

⁹NFL Next Gen Stats via NFL.com, "The coverage that turned the Seahawks into Super Bowl winners and is taking the NFL by storm" (split-safety coverage on 32.9% of dropbacks in 2018 and 42.0% in 2025; Cover 4 at 17.2% in 2025): <https://www.nfl.com/news/the-coverage-that-turned-the-seahawks-into-super-bowl-winners-and-is-taking-the-nfl-by-storm>. The FTN-based figure (43.3% two-high among labelled 2025 regular-season dropbacks) is the course's calculation from nflverse participation data.

The season: Mike Macdonald's Seahawks won Super Bowl LX after a season in which Next Gen Stats, using its tracking-based coverage data, found the league in split-safety (two-high) coverage on 42.0% of dropbacks, the highest rate since its tracking of coverage began in 2018, with quarters (Cover 4) alone at 17.2%.⁹

What to notice: those league numbers come from tracking; the coverage labels in nflverse for 2025 come from FTN's human charters. The two broadly agree (FTN's labels give about 43% two-high for 2025), but they are different measurements of the same thing, and a team's number can differ between them. When a broadcast quotes a coverage rate, ask whose. Then watch Seattle's safeties in the last second before the snap: the two-high to one-high case study shows what Macdonald asks them to do, and this chapter's onset measure is how you would time it.

Three projects to try

The features above are building blocks. Here are three projects that put them together, each with the real labels you would train or test against, a sketch on simulated plays, and the trap that catches most first attempts.

Project 1: man or zone?

The question: from tracking alone, was the coverage man or zone? **The labels:** `pff_manZone` in the Big Data Bowl 2025 `plays.csv`; `team_coverage_man_zone` in the 2026 `supplementary_data.csv`.

The idea: in man coverage a defender's job is a *person*, so he stays close to one receiver wherever that receiver goes. In zone his job is a *place*, so receivers run through his area and away from him. One feature captures much of that: for each defender, find the receiver nearest him at the snap, then measure how far apart they are a second or two later. Call it **stickiness**; small means man.

```
ROUTE_MENU = {"go": r.go(20), "post": r.post(10), "corner": r.corner(10), "out": r.out(10, 6),
              "dig": r.dig(12, 10), "curl": r.curl(11), "comeback": r.comeback(14),
              "slant": r.slant(1.5, 8), "hitch": r.hitch(5), "seam": r.seam(20)}

def jitter(route, rng, sd=1.0):
    """Real receivers don't run textbook depths: wobble each waypoint by about a yard."""
    return [(0, 0)] + [(dn + rng.normal(0, sd), o + rng.normal(0, sd)) for dn, o in route[1:]]

def sim_coverage(kind, rng, play_id):
    """One dropback against Cover 1 ('man') or Cover 3 ('zone'); returns (tracking, routes run)."""
    off, dfn = nickel_vs_2x2(shell="single_high")
    p = Play(off, dfn, los=35, to_go=10)
    ran = {}
    for key in ["X", "H", "Y", "Z"]:
        name = rng.choice(list(ROUTE_MENU))
        p.route(key, jitter(ROUTE_MENU[name], rng))
        ran[key] = name
    p.dropback(2.5)
    if kind == "man":
        # Cover 1: four man defenders, two deep/robber
        for d_, o_ in (("LCB", "X"), ("RCB", "Z"), ("NB", "Y"), ("WILL", "H")):
            p.man(d_, o_)
        p.drop("FS", (16, 0)); p.drop("MIKE", (8, 0)); p.drop("SS", (9, 3))
    else:
        # Cover 3: three deep, four underneath
        j = lambda: rng.normal(0, 1.0)
        # noqa: E731 zone landmarks vary a little
        carry = {"LCB": "X", "RCB": "Z", "NB": "Y"}
        # deep-third corners (and the seam player)
        vertical = {"go", "seam", "post", "corner"}
        # carry a vertical route through their area
        for key, spot in {"LCB": (16, -13), "RCB": (16, 13), "FS": (17, 0), "NB": (8, 12),
                        "SS": (9, 4), "WILL": (7, -11), "MIKE": (9, -3)}.items():
            p.route(key, jitter(ROUTE_MENU[vertical[key]], rng))
```

```

        if key in carry and ran[carry[key]] in vertical:
            p.man(key, carry[key])
        else:
            p.drop(key, (spot[0] + j(), spot[1] + j()))
    return p.to_tracking(fps=FPS, play_id=play_id), ran

def stickiness(t, defenders=("LCB", "RCB", "NB", "WILL")):
    """Median distance (1.0-2.5 s after the snap) from each defender to the receiver nearest him
    at the snap, averaged over defenders. Small = defenders stayed with 'their' receivers."""
    snap = t[np.isclose(t["time"], 0.0)].set_index("displayName")
    later = t[t["time"].between(1.0, 2.5)]
    vals = []
    for d_ in defenders:
        gap = np.hypot(snap.loc[RECEIVERS, "x"] - snap.loc[d_, "x"],
                      snap.loc[RECEIVERS, "y"] - snap.loc[d_, "y"])
        mate = gap.idxmin()
        a_ = later[later["displayName"] == d_].set_index("time")
        b_ = later[later["displayName"] == mate].set_index("time")
        vals.append(np.hypot(a_["x"] - b_["x"], a_["y"] - b_["y"]).median())
    return float(np.mean(vals))

rng = np.random.default_rng(55)
sims, route_rows = [], []
for pid in range(240):
    kind = "man" if pid % 2 == 0 else "zone"
    t, ran = sim_coverage(kind, rng, pid)
    sims.append((pid, kind, stickiness(t)))
    for key, name in ran.items():
        g = t[t["displayName"] == key][["time", "x", "y"]].assign(playId=pid, receiver=key, route=name)
        route_rows.append(g)
mz = pd.DataFrame(sims, columns=["playId", "label", "stickiness"])
cuts = np.linspace(1, 10, 181)
acc = [(mz["stickiness"] < c) == (mz["label"] == "man")).mean() for c in cuts]
best_cut, best_acc = cuts[int(np.argmax(acc))], max(acc)
print(f"best threshold {best_cut:.1f} yd -> {best_acc:.0%} of simulated plays classified correctly")

```

```

best threshold 2.0 yd -> 98% of simulated plays classified correctly

```

```

fig, ax = plt.subplots(figsize=(6.5, 2.8))
bins = np.arange(0, 14.5, 0.5)
for k, lab in enumerate(["man", "zone"]):
    ax.hist(mz.loc[mz["label"] == lab, "stickiness"], bins=bins, color=style.SERIES[k], alpha=0.75,
            label={ "man": "Cover 1 (man)", "zone": "Cover 3 (zone)" }[lab])
ax.axvline(best_cut, color=style.INK_2, lw=1, ls=(0, (3, 2)))
ax.text(best_cut + 0.2, ax.get_ylim()[1] * 0.9,
        f"threshold {best_cut:.1f} yd:\n{best_acc:.0%} correct", fontsize=7.5, color=style.INK_2,
        va="top")
ax.set_xlabel("Stickiness: yards from defender to his receiver-at-the-snap")
ax.set_ylabel("Plays")
ax.legend(loc="upper right", fontsize=8)
plt.show()

```

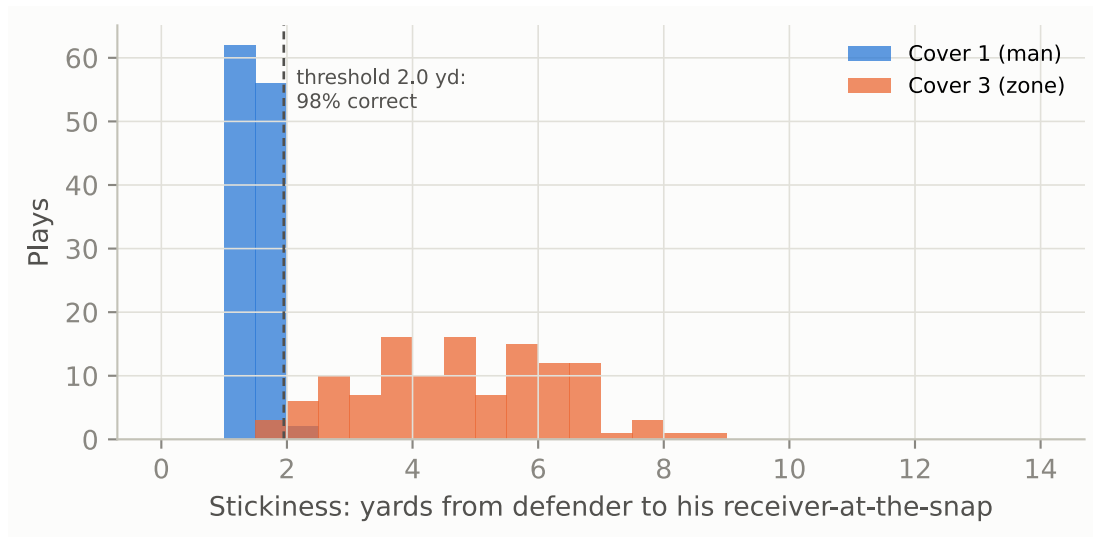


Figure 15: Project 1 on 240 simulated dropbacks (BDB-format), half Cover 1 man and half Cover 3 zone, with random routes. Stickiness is the average distance, one to two and a half seconds after the snap, between each of four underneath and outside defenders and the receiver nearest him at the snap. Man plays (blue) pile up below about 2 yards; zone plays (orange) spread out above, except where several receivers ran vertical routes: Cover 3 corners carry a vertical through their deep third, which looks exactly like man. The dashed line is the single best threshold.

The trap: on simulated plays, where a man defender follows his receiver perfectly, one number sorts 98% of plays correctly. Real football is much harder. Man corners play with leverage and give ground on purpose; zone defenders in pattern-matching coverages *become* man defenders once routes declare; and some plays are man on one side and zone on the other. Expect a real-data version to need more features (orientation toward the receiver or toward the quarterback, how defenders react to motion, the safeties' movement) and a proper model. Above all, **split your training and test data by game, not by play**: plays from the same game share a defense, a game plan and a pair of quarterbacks, so a model tested on plays from games it trained on will look far better than it is.

Project 2: what route did he run?

The question: from a receiver's path alone, name his route. **The labels:** `routeRun` in the Big Data Bowl 2025 `player_play.csv`; in nflverse participation data, the targeted receiver's `route` (charted). **The idea:** standardise each path so that "toward his own sideline" is always positive, then describe it with a few numbers (how deep he got, where he ended up across the field, whether he came back toward the quarterback) and turn the route tree into rules.

```
def route_features(g, los=35.0):
    """Max depth, final depth and final sideways move (+ = toward his own sideline) of one path."""
    g = g.sort_values("time")
    d = g["x"].to_numpy() - g["x"].iloc[0]
    w = g["y"].to_numpy() - g["y"].iloc[0]
    out_sign = 1 if g["y"].iloc[0] < FIELD_WIDTH / 2 else -1      # low-y receivers: out = -y
    out = -w * out_sign
    return pd.Series({"depth_max": d.max(), "depth_end": d[-1], "out_end": out[-1]})

def classify(f):
    """The route tree as if-then rules."""
    came_back = f["depth_max"] - f["depth_end"]
    if came_back > 0.6 and abs(f["out_end"]) < 4.5:      # turned back toward the quarterback
        return "comeback" if f["out_end"] > 1.0 else "curl/hitch"
    if f["depth_max"] >= 16:
        return "corner" if f["out_end"] > 4 else ("post" if f["out_end"] < -4 else "vertical")
    if f["out_end"] > 3:
        return "out"
```

```

if f["out_end"] < -3:
    return "slant" if f["depth_max"] < 8 else "dig"
return "other"

```

```

TRUE_CLASS = {"go": "vertical", "seam": "vertical", "post": "post", "corner": "corner", "out": "out",
              "dig": "dig", "slant": "slant", "curl": "curl/hitch", "hitch": "curl/hitch",
              "comeback": "comeback"}

paths = pd.concat(route_rows)
paths = paths[paths["time"] >= 0]
feats = paths.groupby(["playId", "receiver"]).apply(route_features, include_groups=False)
feats["predicted"] = feats.apply(classify, axis=1)
ran_route = paths.drop_duplicates(["playId", "receiver"]).set_index(["playId", "receiver"])["route"]
feats["truth"] = ran_route.map(TRUE_CLASS)
route_acc = (feats["predicted"] == feats["truth"]).mean()
print(f"{len(feats)} simulated routes; rules agree with the true route on {route_acc:.0%}")

```

960 simulated routes; rules agree with the true route on 88%

```

CLASSES = ["vertical", "post", "corner", "out", "dig", "slant", "curl/hitch", "comeback", "other"]
fig, axes = plt.subplots(2, 5, figsize=(6.5, 3.9), sharex=True, sharey=True)
axes.flat[-1].axis("off")
for ax, cls in zip(axes.flat, CLASSES):
    sub = feats[feats["predicted"] == cls]
    for (pid, rec), f in sub.iterrows():
        g = paths[(paths["playId"] == pid) & (paths["receiver"] == rec)].sort_values("time")
        sgn = 1 if g["y"].iloc[0] < FIELD_WIDTH / 2 else -1
        ax.plot(-(g["y"] - g["y"].iloc[0]) * sgn, g["x"] - g["x"].iloc[0],
                color=style.SERIES[0] if f["truth"] == cls else style.SERIES[1],
                lw=0.6, alpha=0.45 if f["truth"] == cls else 0.9)
    ok = (sub["truth"] == cls).sum()
    ax.set_title(f"{cls}\n{ok} of {len(sub)} right", fontsize=7.5)
    ax.set_aspect("equal")
    ax.tick_params(labelsize=7)
    ax.axhline(0, color=style.BASELINE, lw=0.6)
for ax in axes[1][:4]:
    ax.set_xlabel("to his\ndsideline (yd)", fontsize=7)
for ax in axes[:, 0]:
    ax.set_ylabel("downfield (yd)", fontsize=7.5)
fig.tight_layout()
plt.show()

```

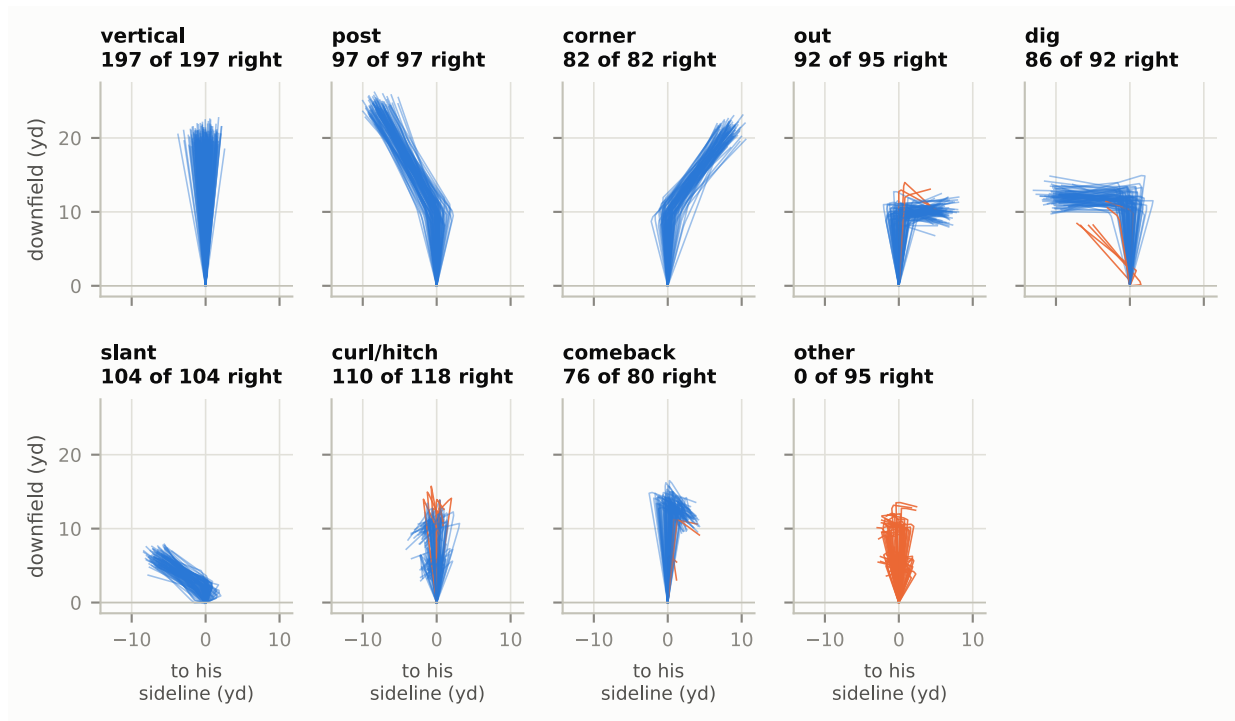


Figure 16: Project 2: 960 simulated routes (BDB-format, each waypoint wobbled by about a yard), drawn from the receiver’s own starting point with his sideline to the right, grouped by the class the rules gave each one. Blue paths match the route the simulation ran; orange ones don’t. The biggest group of misses is ‘other’: curls and hitches that stopped without clearly coming back toward the quarterback, which the rules decline to name. The rest sit on a border between two classes: comebacks that drifted inside and read as curls, slants that ran deep enough to look like digs, outs that sank back like comebacks.

The rules get 88% of the simulated routes right, and the misses are instructive: they cluster at the borders between classes, where one yard of depth or one degree of angle decides the name. Real charters face the same borders and settle them with judgement (“he was supposed to run a curl”) that tracking cannot see. **The trap:** treating the charted label as truth. Two charting services can disagree on a meaningful share of routes; check your model’s “errors” on film before you believe them.

Project 3: measuring disguise

The question: how often does a defense show one shell and play another, and how late does it change? **The labels:** `pff_passCoverage` in the Big Data Bowl 2025 `plays.csv` (the coverage played, charted after the snap). **The idea:** you have already built everything it needs. Classify the shell two seconds before the snap and at the snap (the second-deepest-defender depth from the team chart works: above about 11 yards means two-high), compare both with the charted coverage, and add the rotation onset from step 3. Disguise, Rotation, and Split-Field Coverage shows what the result looks like on eight made-up defenses, and the code for the real files.

The trap: “two-high” is not one thing. A defense can show two-high and play Cover 6 or quarters honestly while its safeties creep and bounce; it can rotate *within* two-high; and charting labels themselves changed between seasons (nflverse participation data switches from NGS to FTN coding in 2023, with different coverage names). Define your classes on paper before you write code, and keep the definition fixed across teams and seasons.

One more, if you want a fourth: expected yards after the catch. The Shanahan wide-zone case study compared nflverse’s expected YAC, which knows only where the ball was caught, with Next Gen Stats’ version, which knows where the defenders were. You can build the second kind yourself: take every completion’s catch frame, describe it with this chapter’s features (the receiver’s speed and direction, the distance and closing speed of the nearest two defenders, the separation at the catch), fit a model of the yards gained after the catch, and subtract. The trap is the same as in Project 1: split by game, and check that what your model calls “over expected” survives a second season.

Project 3 is, nearly word for word, the question the 2025 Big Data Bowl winners answered. Which brings us to the competition itself.

The Big Data Bowl

The **Big Data Bowl** is the NFL's annual public analytics competition. Each year the league releases a slice of tracking data with a theme, and anyone (students, professionals, hobbyists) can enter. The finalists present at the NFL Scouting Combine, and the best ideas tend to show up a year or two later as Next Gen Stats metrics or broadcast graphics. It started in 2019, largely the creation of the league's football data and analytics group led by Michael Lopez, a statistician who had come from academia, and it was designed around a problem the league had: it was sitting on a uniquely rich data set and only a few dozen people inside teams were allowed to work on it.¹⁰ Releasing slices to the public was a recruiting tool as much as a research one. The league says many past participants have since been hired by NFL clubs.

Themes, 2019 to 2026

Edition	Theme	Data released	What it led to
2019	Open questions on speed, routes and coverage	2017 season, first six weeks	Several entries modelled completion probability and "what if" throws to other receivers
2020	Rushing: predict the yards a run will gain from the hand-off	2017-2018 run plays; scored live on 2019 games	The winning model became NGS's expected rushing yards, the basis of RYOE
2021	Defending the pass	2018 season, every pass play	Measures of coverage and defensive back performance
2022	Special teams	2018-2020 punts, kickoffs and kicks	Return and coverage-unit measures
2023	Linemen on pass plays	2021 season, Weeks 1-8	Pass-rush and protection measures from tracking
2024	Tackling	2022 season, Weeks 1-9	Tackle-probability and missed-tackle measures
2025	Pre-snap behaviour, and what it reveals	2022 season, Weeks 1-9, first edition with full pre-snap frames	"Exposing Coverage Tells in the Pre-Snap" won; NGS built a pre-snap disguise model
2026	Player movement while the ball is in the air	2023 season, Weeks 1-18; prediction track scored on late-2025 games	"Ghostbusters" (ghost defenders) won the analytics track
2027	Not announced as of early October 2026		

A few editions deserve a closer look, because their winning ideas changed how the league measures the game.

2020: from a competition to a broadcast stat. The rushing edition was a pure prediction contest: given the tracking frame at the handoff, predict how many yards the run would gain. The winners, Philipp Singer and Dmitry Gordeev, two Austrian data scientists competing as "The Zoo", built a neural network that looked at every pair of an offensive player and a defender (how far apart they were, how fast they were closing), rather than at the players one at a time.¹¹ Next Gen Stats adopted the approach for its expected rushing yards, which is why "rush yards over expected" now appears on broadcasts.

2025: reading the defense before the snap. The first edition with complete pre-snap frames asked what pre-snap movement reveals. The winners, NYU students Smit Bajaj and Vishakh Sandwar, built "Exposing Coverage Tells in the Pre-Snap", a model of

¹⁰NFL Football Operations, "Big Data Bowl": <https://operations.nfl.com/gameday/analytics/big-data-bowl>.

¹¹NFL Football Operations, "2020 Big Data Bowl results": <https://operations.nfl.com/updates/the-game/2020-big-data-bowl-results>; NFL Next Gen Stats Analytics, "Next Gen Stats: Intro to Expected Rushing Yards" (2020): <https://www.nfl.com/news/next-gen-stats-intro-to-expected-rushing-yards>.

¹²NYU School of Professional Studies, "NYU duo crowned winners of the 2025 NFL Big Data Bowl": <https://www.sps.nyu.edu/about/news-and-ideas/articles/preston-robert-tisch-institute-for-global-sport/2025/nyu-duo-crowned-winners-of-the-2025-nfl-big-data-bowl.html>; the NGS disguise

which pre-snap movements give a coverage away; Next Gen Stats then built a disguise model of its own, crediting their work.¹² That is Project 3 above, and the tells catalogue in *Watching Live* is its human version.

2026: the ball in the air. The most recent edition ran two competitions. In the prediction track, entrants were given every player's tracking up to the throw and had to predict where each relevant player would be while the ball was in flight; the leaderboard was scored live on games from Weeks 14 to 18 of the 2025 season. The analytics track was won by Lucca Ferraz of Rice University with "Ghostbusters: Back off man, I'm a Data Scientist!", which built "ghost" defenders, a model of where an average defender would have gone, and graded real defenders against their ghosts.¹³ Notice the release's quirk: the training files are the 2023 season, Weeks 1-18, even though some of the league's announcement text mentions 2023 and 2024.

What the winners have in common

Read enough Big Data Bowl write-ups and a pattern shows up. The entries that win, and the ones the league later adopts, tend to share five habits.

1. **A football question first.** "Which pre-snap movements give away the coverage?" beats "here is a clustering of safety paths." Judges are coaches and league staff; a metric they can't connect to a decision is a curiosity.
2. **An expectation, then a comparison.** Most winning metrics have the shape of RYO: model what an average player would do in this exact situation, then measure the real player against it. Ghost defenders are the same idea in motion.
3. **Relative features.** Positions relative to the ball, the line, the nearest opponent, and speeds relative to each other, not raw x and y . The 2020 winners' pairwise design is the clearest case.
4. **Honest validation.** Does the metric predict something it was not built from? Is it stable from one half of the season to the other? Does it rank players in an order football people recognise, with surprises you can explain?
5. **Pictures.** Nearly every finalist animates plays. An animation of the metric changing frame by frame is how a coach decides whether to trust it, which is why this book's library draws real and simulated plays the same way.

Go deeper

Earlier editions produced ideas you will meet again: a "hypothetical completion probability" for every receiver on a play, not only the one targeted, from the first edition (Sameer Deshpande and Katherine Evans);¹⁴ and from the 2023 linemen edition, STRAIN, a pass-rush measure built from how fast a rusher closes on the quarterback, by Quang Nguyen, Ronald Yurko and Gregory Matthews of Carnegie Mellon.¹⁵ Both were later published as academic papers, which are the best way to see what a winning entry looks like once it has been checked carefully.

model crediting the project: Next Gen Stats Analytics Team, "Next Gen Stats: New advanced metrics you need to know for the 2025 NFL season," NFL.com (2025): <https://www.nfl.com/news/next-gen-stats-new-advanced-metrics-you-need-to-know-for-the-2025-nfl-season>.

¹³Kaggle, "NFL Big Data Bowl 2026 - Prediction" (<https://www.kaggle.com/competitions/nfl-big-data-bowl-2026-prediction>) and "NFL Big Data Bowl 2026 - Analytics" (<https://www.kaggle.com/competitions/nfl-big-data-bowl-2026-analytics>): theme, the two tracks, and the released training files `input_2023_w01..w18.csv` / `output_2023_w01..w18.csv`. Winner and project title per the NFL's Big Data Bowl page, <https://operations.nfl.com/gameday/analytics/big-data-bowl>. As of early October 2026 no 2027 edition had been announced ([kaggle.com/competitions/nfl-big-data-bowl-2027](https://www.kaggle.com/competitions/nfl-big-data-bowl-2027) returned "not found").

¹⁴Sameer K. Deshpande and Katherine Evans, "Expected hypothetical completion probability," *Journal of Quantitative Analysis in Sports* 16, no. 2 (2020): <https://doi.org/10.1515/jqas-2019-0050>.

¹⁵Quang Nguyen, Ronald Yurko and Gregory J. Matthews, "Here comes the STRAIN: Analyzing defensive pass rush in American football with player tracking data," *The American Statistician* (2024): <https://doi.org/10.1080/00031305.2024.2350401>.

💡 Watch for it

Your first tracking chart. You can't download this week's tracking data, but you can practise the whole workflow on a play you watch live, and check part of it against real numbers within days.

- **Pick one play** during the game, ideally a dropback with motion. Write down the quarter, the clock, the down and distance, and three facts: the shell before the snap (count the safeties deeper than about 10 yards), whether anyone was moving at the snap, and where the ball went.
- **Watch one safety** from the moment the offense sets until the throw. Did he move before the snap or at it? Say "now" out loud when he starts to rotate; you are estimating rotation onset by ear.
- **After the game**, find the play in nflverse play-by-play (`load_pbp()` for the current season, matched on quarter and clock) to get the official description and the target.
- **Draw it** in `gridiron` the way you remember it: formation, motion, the routes you saw, the safety rotation. Export it with `to_tracking()` and run this chapter's `separation()` and `safety_tracks()` on it.
- **Check what you can.** Next Gen Stats' weekly receiving numbers (`load_nextgen_stats(stat_type= "receiving")`, weekly rows) include the targeted receiver's average separation and cushion for that game. Was your play typical for him?
- **Keep the file.** When that season's tracking appears in a future Big Data Bowl, run the same code on the real frames and see how good your eyes were.

Predict the play

The pictures use the course's position letters: on offense X, Z and H are wide receivers, Y the tight end, R the back and Q the quarterback; on defense E is an end, T a tackle, M and W the middle and weak-side linebackers, C a cornerback, \$ the nickel back, and FS and SS the safeties.

💡 Predict the play

Motion, and nobody follows. The offense is in the same 2x2 set as the chapter play and sends H across the formation from the left slot. The two frames below are 2 seconds before the snap and the snap itself, with faint trails showing where everyone has moved. Before the snap: is the defense more likely in man or zone? What would this chapter's motion and travel measures say? And with two safeties still at 13 yards at the snap, what do you still *not* know?

```
def zone_motion_play():
    off, dfn = nickel_vs_2x2()
    p = Play(off, dfn, los=35, to_go=10)
    p.motion("H", to=(-2.2, 3.5), speed=6.0)
    p.motion("WILL", to=(5.0, -3.5), speed=1.6) # bumps toward the motion, two or three yards
    p.motion("MIKE", to=(4.5, 3.6), speed=1.2)
    p.motion("NB", to=(5.0, 10.6), speed=0.5)
    p.dropback(2.0)
    return p

show_animation(zone_motion_play(), times=[-2.0, 0.0])
```

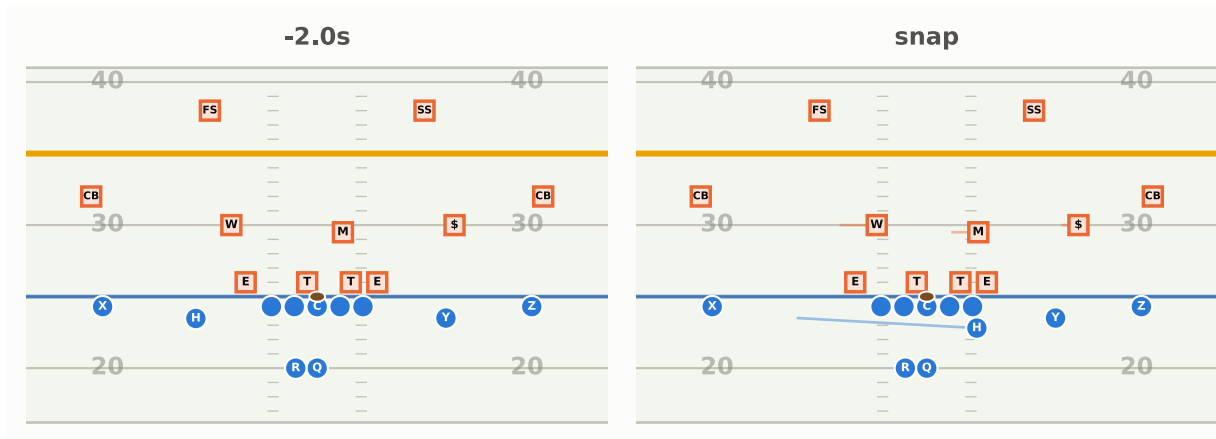


Figure 17: Drill 1 (simulated, BDB-format): H in motion from the left slot to the right, 2 seconds before the snap and at the snap. Watch the defenders' trails, not the motion man.

Answer 1 is at the end of the chapter. Try it first.

💡 Predict the play

Read the chart. Here are the two deepest defenders on one simulated play, measured exactly as in step 1 of the rotation section. Before the snap, what shell did the defense show? What did it play after the snap? Did the rotation start early enough for the quarterback to use it, and what would `rotation_onset()` return?

```
rng_d = np.random.default_rng(31)
t_d = sim_dropback("spin2", rng_d, tau=-0.8)
s_d = safety_tracks(t_d, los=35, ball_y=t_d["ball_y"].iloc[0])
fig, axes = plt.subplots(2, 1, figsize=(6.5, 3.6), sharex=True)
PANELS = ((0, "depth", "Depth (yd)"), (1, "from_middle", "From the middle (yd)"))
for i_, col, ylab in PANELS:
    ax = axes[i_]
    ax.axvspan(-30, 0, color=style.GRID, alpha=0.35, lw=0)
    ends = s_d[s_d["frame_rel"] == 20].set_index("displayName")[col]
    for k, (key, g) in enumerate(s_d.groupby("displayName")):
        ax.plot(g["frame_rel"], g[col], color=style.SERIES[k + 1], lw=2.2)
        close = abs(ends.iloc[0] - ends.iloc[1]) < 1.2 # keep the two end labels apart
        nudge = (0.6 if ends[key] == ends.max() else -0.6 if close else 0.0)
        ax.text(20.6, g[col].iloc[-1] + nudge, "A" if k == 0 else "B", color=style.SERIES[k + 1],
               fontsize=8, fontweight="bold", va="center")
    ax.axvline(0, color=style.INK_2, lw=1)
    ax.set_ylabel(ylab, fontsize=8.5)
    ax.set_xlim(-30, 24)
axes[1].set_xlabel("Frames from the snap (10 frames = 1 second)", fontsize=8.5)
fig.tight_layout()
plt.show()
```

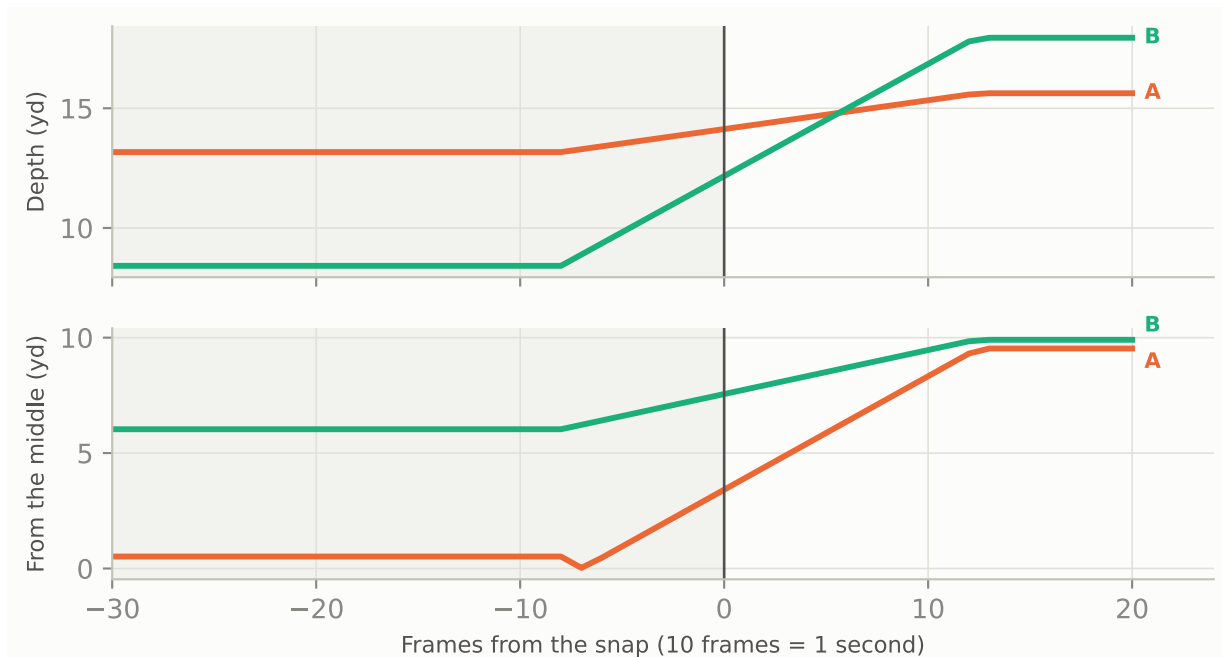


Figure 18: Drill 2: the two deepest defenders on one play (simulated, BDB-format), 3 seconds before to 2 seconds after the snap. Top: depth. Bottom: sideways distance from the middle of the field.

Answer 2 is at the end of the chapter. Try it first.

💡 Predict the play

Read the rows. One frame from the chapter play, 0.7 seconds after the snap, with three players marked A, B and C. Their rows from the standardised file are in the table under the picture. Which one is backedpedalling, which is running a route, and which is standing still? For which of them should you ignore `dir`?

```
mystery = {"LCB": "A", "H": "B", "RB": "C"}
now = one[one["frame_rel"] == 7].copy()
now["label"] = now["displayName"].map(mystery).fillna("")
fig, ax = plt.subplots(figsize=(6.5, 4.8))
fld = Field(ax=ax, orient="vertical", los=L0S, window=(-10, 19), to_go=10,
            lateral=(BALL_Y - 20, BALL_Y + 20))
draw_frame(fld, now)
for key in mystery:
    row = now[now["displayName"] == key].iloc[0]
    px, py = fld.xy(row["x"], row["y"])
    ax.add_patch(plt.Circle((px, py), 1.4, fc="none", ec=style.THIRD, lw=2, zorder=9))
plt.show()
```

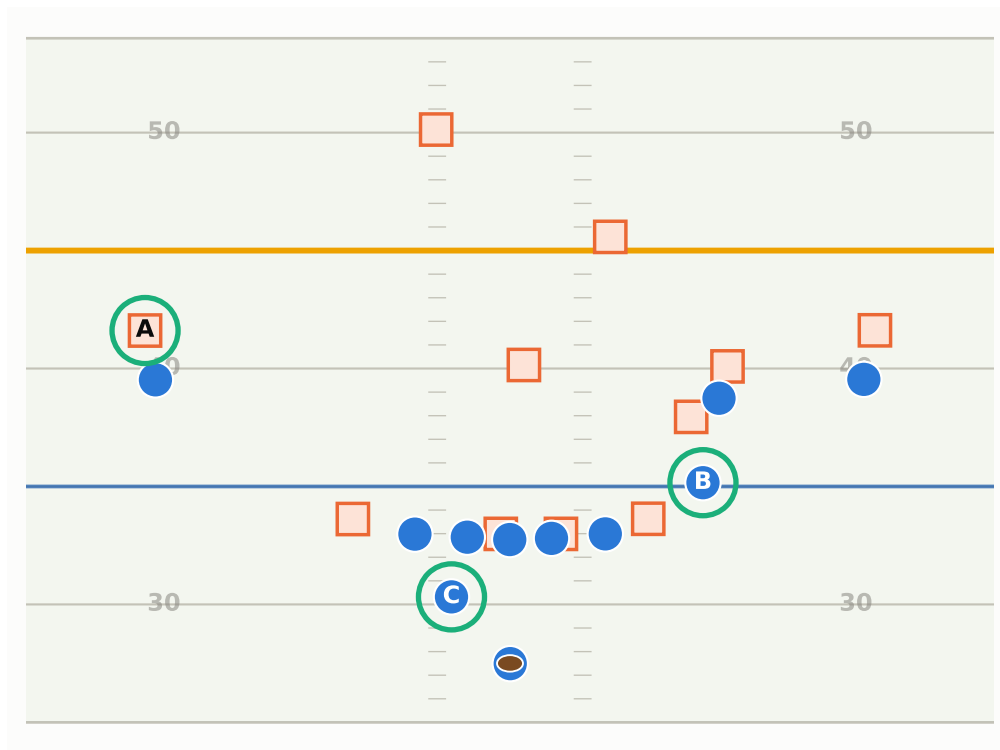


Figure 19: Drill 3 (the chapter play, simulated, BDB-format), 0.7 seconds after the snap. Three players are marked A, B and C (green rings); everyone else is unlabelled.

```
rows_abc = now[now["displayName"].isin(mystery)].assign(player=lambda t: t["displayName"].map(mystery))
md_table(rows_abc.sort_values("player")[["player", "x", "y", "s", "o", "dir"]], 1)
```

player	x	y	s	o	dir
A	51.6	42.1	3.2	272.5	100.3
B	45.2	18.5	7.5	149	153.9
C	40.3	29.1	0.5	83.8	24.1

Answer 3 is at the end of the chapter. Try it first.

Takeaways

- **A tracking file is one row per player per frame**, ten frames a second, on a 120×53.3 -yard grid whose origin is a corner of the field. `s` and `a` are in yards per second ($\times 2.05$ for mph) and yards per second², `o` is where a player faces and `dir` where he moves, both in degrees clockwise from +y.
- **Standardise and align before anything else**: rotate left-going plays by 180 degrees (flip `x` and `y`, turn both angles), then number frames from the event your question is about, the snap or the throw.
- **The same library draws real and simulated plays**. `bdb.prepare()` turns a raw file into frames; `show_animation()` and `frame_strip()` draw them; this book's `d` and `w` are just standardised `x` and `y` measured from the ball.
- **Features are simple geometry with hidden choices**: separation is “nearest defender” (not the covering one) and depends on when you measure it; pocket area ignores who is blocked; motion needs a speed threshold; “safeties” are best defined as the two deepest defenders before the snap. Season-long separation is mostly a route-depth stat.
- **Rotation is a divergence in time**, and rotation onset turns “late rotation” into a number per play. Disguise is two numbers per defense: how often the shell lies, and how late it changes.
- **The Big Data Bowl** has released a new slice of tracking data with a new theme every year since 2019. Its best ideas (expected rushing yards, pre-snap disguise models, ghost defenders) share a shape: a football question, an expectation, a comparison, honest validation and a picture.

Connections

Builds on: Tendencies: PROE, Personnel, Motion, and Coverage (the charting labels that tracking is compared with) and Disguise, Rotation, and Split-Field Coverage (what the safety-rotation chart means). The pocket hull comes from Pass Protection, get-off and time to pressure from The Pass Rush, and RYOE from Expected Points, EPA, and Success Rate.

Sets up:

- Fangio's Two-High Revolution and Macdonald's Answer: the defenses this chapter's rotation chart was built to read.
- Watching Live: the full catalogue of pre-snap tells, the human version of the 2025 Big Data Bowl.
- Film Study and Charting: charting your own games, and scoring your predictions against what happened.
- Scheme and Fantasy: separation and target depth as usage signals.

If you want to go deeper: the Big Data Bowl pages on Kaggle (each edition's data description and the winning notebooks are public); Next Gen Stats' own site, nextgenstats.nfl.com, for the published metrics and their definitions; Michael Lopez's introduction to the *Journal of Quantitative Analysis in Sports* special issue on tracking data, a short, readable map of the field;¹⁶ and the academic versions of winning entries cited above. For working code, the public Big Data Bowl notebooks on Kaggle are the fastest way to see how other people standardise, smooth and feature-engineer the same files.

Answers to Predict the play

Answer 1.

Zone is the better bet. Nobody travelled with H: the Will linebacker who was over him slid two or three yards toward the middle and the Mike slid over, but no defender crossed the formation with him. In this chapter's terms, every defender's “yards moved across since motion began” stays small while H's grows past 12, so the travel feature says *no man defender followed*. The two linebackers bumping over together is what zone defenses do to keep their areas balanced to the new strength.

What you still don't know is the shell after the snap. Two safeties at 13 yards at the snap fit Cover 2, quarters or Cover 6, but they also fit a late rotation to Cover 3. Motion answered “man or zone?”; only the first frames after the snap answer “which zone?”. That is exactly why defenses that rotate late pair it with zone-looking motion responses: the motion tell and the shell tell can be made to point different ways.

Answer 2.

```
pre = s_d[s_d["frame_rel"] == -20].set_index("displayName")
post = s_d[s_d["frame_rel"] == 20].set_index("displayName")
for when, df in (("2 s before the snap", pre), ("2 s after the snap ", post)):
```

¹⁶Michael J. Lopez, “Bigger data, better questions, and a return to fourth down behavior: an introduction to a special issue on tracking datasets in sports,” *Journal of Quantitative Analysis in Sports* 16, no. 2 (2020): <https://doi.org/10.1515/jqas-2020-0057>.

```
print(when, "(depth, from middle):", df[["depth", "from_middle"]].round(1).values.tolist())
print("rotation_onset():", rotation_onset(s_d))
```

```
2 s before the snap (depth, from middle): [[13.2, 0.5], [8.4, 6.0]]
2 s after the snap (depth, from middle): [[15.6, 9.5], [17.9, 9.9]]
rotation_onset(): None
```

One-high shown, two-high played. Before the snap one defender sits deep in the middle (about 13 yards deep, within a yard of the middle) and the other is down at about 8 yards, 6 yards off to one side: a one-high shell. After the snap both are 15 to 18 yards deep and about 10 yards either side of the middle: two deep halves, so a two-high coverage (Cover 2 or quarters). This is a “spin” the other way from the chapter play, one-high to two-high.

The movement starts about **eight frames before the snap**, so a quarterback watching the safeties could see the shell changing during his cadence: not a late rotation. And `rotation_onset()` returns `None`, because it was written to find rotations *to* one-high (one safety finishing in the middle). That is a useful reminder about home-made features: they answer exactly the question you coded, so a play that moves the other way slips through silently. Extending it to spins toward two-high is a good first edit.

Answer 3.

A is backpedalling. He faces the backfield (`o` near 270 degrees, toward $x = 0$) while moving downfield (`dir` near 90): the left corner, giving ground in off coverage before he turns to run with X.

B is running a route. He faces and moves the same way, at about 7 to 8 yards a second, heading toward the offense’s right (`dir` well above 90 means he is drifting toward low y , the offense’s right): H, who was in motion at the snap and is now on his way to the flat.

C is standing still, or close to it: his speed is a fraction of a yard a second. His `o` is meaningful (he faces downfield, set to block), but his `dir` is just the direction of the chips’ wobble and could be anything. That is the running back, who stepped beside the quarterback and is waiting for a blitz who never came. Any feature built on `dir` should drop rows like C’s: filter on `s` first.